

A FIELD GUIDE FOR CISOs, INCIDENT RESPONDERS AND PLATFORM ENGINEERS

BREAKING THE CHAIN



The Anatomy, Lessons and Defense of Software Supply Chain
Attacks

35 incidents · 18 bypassed defences · 73 controls · Aug 2025 – Sep 2026

VINOTHKUMAR R · ARAVIND BASKARAN



BREAKING THE CHAIN

The Anatomy, Lessons and Defense of Software Supply Chain
Attacks

Vinothkumar R
Aravind Baskaran

October 2026

Breaking the Chain: The Anatomy, Lessons and Defense of Software Supply Chain Attacks

© 2026 Vinothkumar R and Aravind Baskaran. All rights reserved.

Edition: October 2026. Incident record current to 3 October 2026.

This book combines the authors' incident notes with public reporting by security vendors, registries and government agencies. Several causal links between incidents are reported or probable rather than confirmed, and the text says so. Package counts, losses and exposure windows are given as first reported and may since have been revised. Arrests mentioned are allegations and charges, not findings of guilt.

Product and project names identify categories of control and are not endorsements. Nothing in this book is legal advice; confirm regulatory obligations with counsel.

About the authors



Vinothkumar R

MANAGER, INCIDENT RESPONSE AND ENTERPRISE IT SECURITY, FRESHWORKS

Vinothkumar is a cybersecurity leader with a decade of experience across Incident Response, Security Operations, Threat Hunting, and Enterprise Security. He leads Incident Response and Enterprise IT Security at Freshworks, managing high-impact security incidents, building scalable security capabilities, and partnering with engineering and executive leadership to reduce risk and protect business continuity.

Previously a Security Engineer at Microsoft, he brings deep experience in advanced incident response and threat hunting across large-scale cloud and enterprise environments, complemented by consulting experience at EY and Deloitte. He works closely with boards and senior leadership to translate complex cyber risk into clear business impact, strategic priorities, and investment decisions.

LinkedIn [linkedin.com/in/vinothkumar-incidentresponse](https://www.linkedin.com/in/vinothkumar-incidentresponse)



Aravind Baskaran

SENIOR SECURITY ENGINEER, LINKEDIN

Aravind Baskaran is an experienced information security professional specialising in incident response, threat hunting, digital forensics, and security engineering. He brings diverse industry experience from Microsoft, Imperva, Deloitte, and LinkedIn, with strong expertise in Azure Cloud Security, Azure DevOps, and digital forensics. He focuses on anticipating emerging threats and strengthening the resilience of modern organisations.

LinkedIn [linkedin.com/in/aravind-baskaran-6779b6b6](https://www.linkedin.com/in/aravind-baskaran-6779b6b6)

Contents

About this book	8
Part I: Foundations	11
Chapter 1. The modern software supply chain.....	12
Chapter 2. Anatomy of a credential-relay campaign	14
Part II: The incidents	17
Chapter 3. The Nx → Shai-Hulud lineage (August – December 2025).....	18
Chapter 4. GlassWorm and the IDE extension attack surface (October 2025 – April 2026)	22
Chapter 5. The TeamPCP / Trivy cascade (February – April 2026)	24
Chapter 6. Mini Shai-Hulud, TanStack, Miasma and ChainDrop (April – September 2026).....	30
Chapter 7. Independent actors and the AI ecosystem (June – September 2026)	37
Part III: Analysis	41
Chapter 8. The prevention methods that were bypassed	42
Chapter 9. The security controls that were missed	44
Part IV: Building the programme from scratch.....	47
Chapter 10. Principles, ownership and the maturity model.....	48
Chapter 11. Preventive controls	50
Chapter 12. Detective controls	56
Chapter 13. Response readiness.....	59
Chapter 14. Implementation roadmap and metrics	61
Chapter 15. Master recommendation sheet	63
Appendices	67
Appendix A. Incident timeline	68
Appendix B. Glossary	69
Appendix C. Mapping to common frameworks	70
Appendix D. Sources and further reading.....	71

About this book

Between August 2025 and September 2026, at least 35 publicly reported incidents turned trusted software into a delivery channel for credential theft. Most of them were connected: secrets stolen in one incident became the key that opened the next. This book documents those incidents, names the defences each one defeated, and turns the lessons into a control programme you can build from nothing.

Who this book is for

It is written for CISOs, security architects, incident responders, platform engineers and open-source maintainers. A reader with no supply chain programme should be able to finish Part IV with a sequenced plan. A reader with a mature programme should be able to use Parts II and III to test whether their controls would have held.

How the book is organised

Part	Chapters	What it gives you
I. Foundations	1–2	A map of the software supply chain and the credential-relay pattern that connected the year's attacks
II. The incidents	3–7	Every incident in a fixed template: what happened, the entry point, the payload, the prevention bypassed, the controls missed, and the detection opportunity
III. Analysis	8–9	The prevention methods attackers walked around, and a matrix of missed controls across all incidents
IV. Building the programme	10–15	Principles, preventive controls, detective controls, response readiness, a phased roadmap and a master recommendation sheet
Appendices	A–D	Timeline, glossary, framework mapping and sources

How each incident is written up

Every incident in Part II uses the same six headings so they can be compared side by side. **What happened** is the narrative. **Entry point** is the first weakness the attacker used. **Payload and behaviour** describes what the malicious code did. **Prevention bypassed** names the defences that were present, or commonly assumed, and failed. **Controls missed** names the controls that would have stopped or contained it. **Detection opportunity** names where a defender could have seen it early.

Scope and evidence

The window runs from the Nx “s1ngularity” compromise on 26 August 2025 to the MemTensor compromise on 23 September 2026. The two 2025 Nx and Shai-Hulud waves sit slightly outside a strict twelve-month window; they are included because later incidents trace back to them.

The incident record combines the authors' own incident notes with public reporting from security vendors, registries and government advisories, current to 3 October 2026. Several causal links between incidents are reported or probable rather than confirmed, and the text says so where that is the case. Package counts, loss figures and exposure windows are given as first reported and may since have been revised.

What this book does not do

It does not attribute attacks beyond what public reporting supports. Arrests mentioned in the text are allegations and charges, not findings of guilt. It does not reproduce malware, exploit code or working indicators beyond what is needed to explain a control. Product names appear as examples of a control category, not endorsements.

PART I

Foundations

CHAPTER 1

The modern software supply chain

A software supply chain is every person, account, system and artefact that sits between a line of code and the machine that runs it. In the incidents in this book, attackers never needed a bug in the victim's application. They needed one trusted step in that chain: a maintainer's token, a CI workflow, a container tag, an editor extension.

1.1 The six attack surfaces

The year's incidents landed on six surfaces. Each is trusted by default, each executes code automatically, and each holds or can reach credentials.

Surface	What it includes	Why attackers want it	Incidents in this book
Source hosting and CI/CD	GitHub, GitLab and CircleCI repositories, organisations, workflows, reusable actions, runners, caches	Runners hold the most privileged secrets an organisation has, and workflows run automatically on events an outsider can trigger	Nx, Trivy, Checkmarx KICS, TanStack, Actions-cool, Miasma, Microsoft orgs, MemTensor
Language package registries	npm, PyPI, RubyGems, crates.io, Go modules, Packagist	One publish reaches every consumer; install and import hooks run code without a prompt	Shai-Hulud, CanisterWorm, LiteLLM, Telnix, AntV, ChainDrop, Mastra and most others
Container and artefact registries	Docker Hub, GHCR, ECR, GitHub Releases	Images and binaries are pulled by tag and run inside CI and production	Trivy images, KICS images, Cemu release assets
Developer endpoints	Laptops and workstations, shells, local package caches, password managers	Developers hold long-lived tokens for everything they can publish or push	Nx, Shai-Hulud, Bitwarden CLI, Nx Console
IDE extensions and AI coding tools	VS Code Marketplace, OpenVSX, AI agent CLIs, agent hook and settings files in repositories	Extensions run with the user's full privileges; AI agents run commands and read files when a repository is opened	GlassWorm, Trivy extension, Nx Console, Microsoft orgs, Miasma Go modules
Third-party and SaaS integrations	OAuth apps, chat and monitoring webhooks, browser extension stores	Integrations carry tokens that are rarely inventoried and rarely rotated	Trust Wallet, CrowdSec downstream breach

1.2 The trust assumptions attackers exploited

Every surface rests on an assumption that held for years and broke in this period. Four assumptions failed repeatedly.

“A package from a known maintainer is safe.” Most malicious versions in this book were published from the real maintainer's account, under the real package name. Reputation checks, typosquat detection and allow-lists of package names did nothing against them.

“A version tag points to the code I reviewed.” Git tags and container tags can be moved. In the Trivy compromise, 75 of 76 tags on one widely used GitHub Action were repointed to malicious commits; every pipeline that referenced a tag ran the new code.

“Provenance proves a release is genuine.” Provenance proves which workflow built an artefact. In Miasma and ChainDrop the attacker controlled that workflow, so malicious releases carried valid SLSA provenance and signed attestations.

“Secrets in CI are protected by masking.” Log masking hides secrets from output. It does not stop a process on the runner from reading them, and several payloads read secrets directly from the runner's process memory.

1.3 Where code runs without anyone choosing to run it

The defining feature of this period is the number of places code executes automatically. A defender who only watches one of them will miss the rest.

Execution point	Ecosystem	Triggered by	Seen in
preinstall / postinstall scripts	npm, pnpm, Yarn, Bun	Any install, including in CI and in dependency updates	Shai-Hulud, Shai-Hulud 2.0, CanisterSprawl, Bitwarden CLI, AntV, Mastra
binding.gyp build steps	npm native modules	Install of a package with native build	Miasma
.pth files	Python	Every start of the Python interpreter	LiteLLM, Hades, CanisterSprawl
Import-time code	Python, JavaScript	First import of the library, including in tests	Telnyx, MemTensor
build.rs scripts	Rust	cargo build	Cross-registry campaigns reported in 2026
Workflow triggers	GitHub Actions, CircleCI	Push, pull request, fork PR, tag, cache restore	Nx, Trivy, TanStack, SAP, Miasma
Editor and agent hooks	VS Code tasks, extension activation, AI agent settings files	Opening a folder or repository	GlassWorm, Nx Console, Microsoft orgs, Miasma Go modules

1.4 What a supply chain programme must protect

A useful programme protects four things, in this order of priority. First, the credentials that can publish, push or deploy. Second, the build systems that hold those credentials. Third, the intake of third-party code into builds and endpoints. Fourth, the integrity of what the organisation itself ships to others.

The chapters that follow show that nearly every incident began with the first item. Part IV is therefore organised around credentials first and artefacts second.

CHAPTER 2

Anatomy of a credential-relay campaign

The year's attacks were not 35 separate events. They were three or four relays in which each victim's stolen secrets funded the next compromise. Understanding the relay is the single most useful idea in this book, because it changes what “contained” means.

2.1 The relay loop

Every major campaign ran the same five-step loop.

- 1. Gain one publishing or push credential.** The first credential came from a misconfigured workflow (Nx, Trivy, TanStack), a forked-PR secret leak (SAP), a phished maintainer (chalk/debug) or a token that was never revoked (the Trivy OpenVSX token, Mastra).
- 2. Plant a payload where it runs automatically.** Install hooks, import-time code, .pth files, moved tags, poisoned caches, editor and agent hooks.
- 3. Harvest every secret the payload can reach.** Environment variables, configuration files, cloud metadata, password manager vaults, and in the most advanced cases the CI runner's process memory.
- 4. Exfiltrate somewhere hard to block.** Public GitHub repositories under the victim's own account, typosquatted domains, and decentralised infrastructure such as blockchain-hosted canisters and smart contracts.
- 5. Reinvest.** Use the stolen npm, PyPI, Docker Hub or GitHub tokens to publish the next malicious version, then return to step 2 in a new organisation.

Worms automate step 5 inside the payload: Shai-Hulud, CanisterWorm, CanisterSprawl, ChainDrop and the MemTensor skit binary all republished themselves with whatever tokens they found. Campaigns without worm code ran step 5 by hand, often weeks later.

2.2 The three relays of the year

Relay	Seed	Downstream victims (examples)	Span
Nx → Shai-Hulud	Nx pull_request_target workflow, 26 Aug 2025	Shai-Hulud, Shai-Hulud 2.0, Trust Wallet	Aug–Dec 2025
TeamPCP / Trivy	Stale OpenVSX token and a pwn request on Trivy CI, 27–28 Feb 2026	CanisterWorm, Docker Hub images, Checkmarx KICS, LiteLLM, Telnyx, CanisterSprawl, Bitwarden CLI	Feb–Apr 2026
Mini Shai-Hulud / TanStack	CircleCI forked-PR secrets at SAP, 29 Apr 2026; TanStack cache poisoning, 11 May 2026	Cemu, UiPath, Mistral, Nx Console, AntV, DurableTask, Miasma, Microsoft orgs, ChainDrop, CrowdSec	Apr–Sep 2026

Public reporting links ChainDrop and the Mini Shai-Hulud wave to the same actor tracked as TeamPCP. Two incidents in this book, Mastra and MemTensor, sit outside these relays and show the same techniques spreading to other actors.

2.3 How long a stolen credential stays useful

Of 15 downstream incidents where the original theft can be dated, 10 were abused within 8 days. The median lag was 8 days and the longest was 34 days, when Docker Hub and npm credentials from the 19 March Trivy wave were used on 22 April. A CrowdSec breach disclosed in September 2026 used an OAuth token from the May wave, months after the original theft.

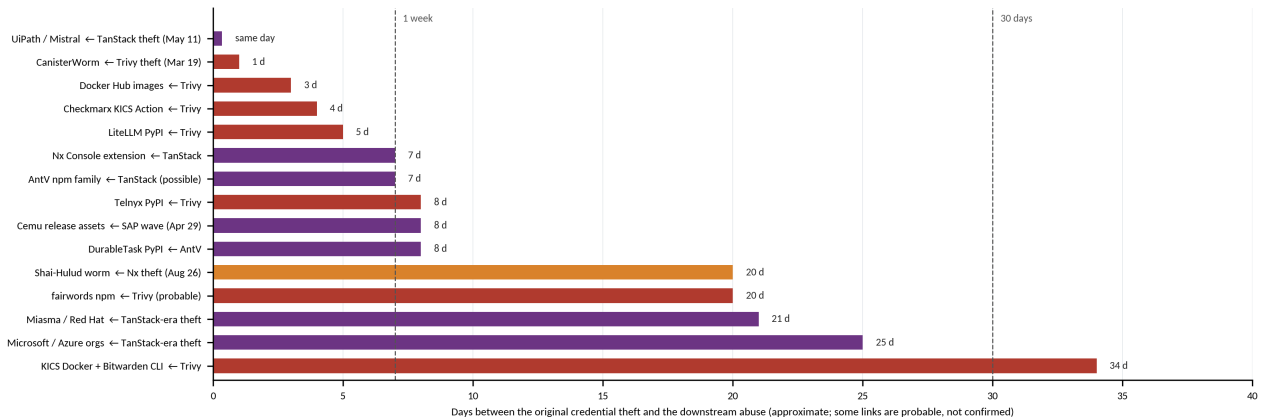


Figure 2.1. Days between each original credential theft and its downstream abuse, for 15 traceable links. Colours follow the relays in section 2.2: orange Nx → Shai-Hulud, red TeamPCP / Trivy, purple Mini Shai-Hulud / TanStack. Some links are probable rather than confirmed.

The chart carries one operational lesson. Containment that is declared within a day but leaves even one credential live has not contained anything; the attacker simply waits. Chapter 11 turns this into the Atomic Rotation control, and Chapter 12 into a 90-day watch on retired credentials.

2.4 Why defenders kept losing the race

Three features of the relay defeated ordinary incident response.

The victim is not the only owner of the stolen secret. A contributor's token stolen on a personal laptop gave access to an employer's organisation (Microsoft, Red Hat, Nx Console). The organisation that owned the repository had no telemetry on the machine that leaked it.

Containment was scoped to the alert, not the credential. Aqua Security rotated credentials after the first Trivy breach, but not all of them. Checkmarx cleaned up its March compromise, but a Docker Hub token survived until April.

Attackers targeted the responders. Spam floods buried disclosure threads. ChainDrop wiped home directories when a stolen token was revoked. Hades carried prompt-injection text aimed at AI-assisted analysis. A response plan that does not expect this will stall at the moment it matters most.

PART II

The incidents

CHAPTER 3

The Nx → Shai-Hulud lineage (August – December 2025)

One injectable GitHub workflow at Nx in August 2025 produced the credentials that seeded the first self-replicating npm worm three weeks later. By December the same family had reached a browser wallet extension and roughly US\$8.5 million in user funds. The lesson of this chapter is that a single CI misconfiguration can become an ecosystem event.

3.1 Nx “s1ngularity”, 26–29 August 2025

Ecosystem	Relay	Scale reported
npm (via GitHub Actions)	Seed of the Nx → Shai-Hulud relay	Malicious versions of Nx build-system packages; public reporting cites 2,349 credentials from 1,079 developer systems

What happened. Attackers published malicious versions of packages in the Nx build-system family to npm. Developers and CI systems that installed them during the window ran a credential stealer.

Entry point. A workflow triggered by `pull_request_target` placed the pull-request title directly into a shell command. Because `pull_request_target` runs in the context of the base repository, the workflow held an elevated token. Attacker-controlled title text became a command, and the attacker used the resulting access to reach the npm publishing credential.

Payload and behaviour. On install, the payload searched for GitHub and npm tokens, SSH keys, `.env` files, cloud credentials and cryptocurrency wallet data. It looked for locally installed AI command-line tools and invoked them to help with reconnaissance, an early sign of attackers using the victim's own AI agents. Stolen data was uploaded to public repositories created under the victim's GitHub account, named with an “s1ngularity” prefix. Some victims' private repositories were then made public with the stolen credentials.

Prevention bypassed. Code review did not help, because the attacker never changed code; a pull-request title was the exploit. Branch protection did not help, because the workflow ran from the protected base branch. npm account two-factor authentication did not help, because publishing used a stored token rather than an interactive login.

Controls missed.

- Untrusted event data (titles, branch names, commit messages) was interpolated into run steps instead of being passed through environment variables.
- A `pull_request_target` workflow had more than read-only token permissions.
- The npm publishing credential was a long-lived stored token rather than a short-lived trusted-publishing (OIDC) exchange bound to one release workflow.
- The publish secret was not isolated in a protected environment requiring approval.
- Consumers ran install scripts by default and resolved new versions immediately.
- Developer endpoints let a package install launch AI agents with permissive flags.

Detection opportunity. A workflow run from a first-time contributor's pull request with shell metacharacters in its title; an npm publish that did not come from the project's normal release workflow; endpoint telemetry showing a Node.js install process spawning an AI CLI; new public repositories with the campaign's naming pattern appearing in employees' accounts.

3.2 chalk / debug maintainer takeover, 8 September 2025

Ecosystem	Relay	Scale reported
npm	Independent	A group of extremely widely used utility packages; exposure measured in hours

What happened. A prolific maintainer of foundational npm utilities, including chalk and debug, was phished. The attacker published malicious versions of the maintainer's packages.

Entry point. A convincing “account security” email led the maintainer to a look-alike login page that captured the password and the one-time two-factor code in real time.

Payload and behaviour. The injected code targeted browser contexts. It hooked wallet and network functions to silently redirect cryptocurrency transactions to attacker-controlled addresses. The community detected the change within hours, which limited losses.

Prevention bypassed. Time-based one-time-password two-factor authentication was present and was phished along with the password. Package reputation and download counts gave no signal, because the packages were genuine.

Controls missed.

- Phishing-resistant multi-factor authentication (passkeys or hardware security keys) on the registry account.
- Publishing restricted to a CI release workflow with provenance, so that a stolen login alone cannot publish.
- On the consumer side, a minimum release age before new versions are accepted, and lockfiles that pin exact versions.

Detection opportunity. A new version published outside the maintainer's usual release workflow and without the provenance attestation earlier versions carried; a sudden minified code block added to a package whose diff is normally small.

3.3 Shai-Hulud, 14–16 September 2025

Ecosystem	Relay	Scale reported
npm	Nx → Shai-Hulud	Hundreds of packages (around 500 per contemporary government reporting)

What happened. The first self-replicating worm in the npm registry. Each infected maintainer's packages were re-published with the worm inside, infecting the next set of developers and CI systems.

Entry point. Stolen npm and GitHub secrets. The authors' notes trace the seed to credentials harvested in the Nx wave; other reporting also describes phished maintainers. In either case, the first publish used a valid maintainer token.

Payload and behaviour. A postinstall script ran a bundled payload that used the open-source TruffleHog scanner to search the machine for secrets: npm tokens, GitHub personal access tokens and AWS, GCP and Azure keys. Where it found an npm token, it listed the packages that token could publish, injected itself and republished up to 20 of them. Stolen data went to public GitHub repositories named “Shai-Hulud.”

Prevention bypassed. Registry two-factor authentication did not apply to automation tokens. Lockfiles did not protect projects that ran fresh installs or automated dependency updates. Malware scanning at the registry did not stop republished versions of real packages quickly enough.

Controls missed.

- npm accounts configured to require two-factor authentication for publishing and to disallow token publishing, or publishing moved to trusted publishing.
- Granular, short-lived, package-scoped tokens instead of broad tokens that could publish every package a maintainer owned.
- No long-lived registry tokens stored in `~/.npmrc` on developer machines.
- Install scripts disabled by default for dependencies, with an explicit allow-list.
- Endpoint detection for secret-scanning tools executed by a package install.

Detection opportunity. TruffleHog or a similar scanner spawned by npm or node; a burst of patch-version publishes across many packages from one maintainer token; creation of repositories called “Shai-Hulud” under employee accounts, which GitHub search and secret-scanning partner alerts surfaced within hours.

3.4 Shai-Hulud 2.0, “The Second Coming”, 21–24 November 2025

Ecosystem	Relay	Scale reported
npm	Nx → Shai-Hulud	More than 1,000 packages reported across the wave

What happened. A second, larger wave of the same worm, rebuilt to run earlier, evade scanners and persist.

Entry point. Credentials harvested in earlier waves that had not been rotated.

Payload and behaviour. The worm moved from postinstall to preinstall, so it ran before any later install step could inspect it. A two-stage loader downloaded the Bun JavaScript runtime and ran the payload under Bun, evading tools that watch only Node.js. It raised its propagation limit from 20 to 100 packages per run. For persistence it registered compromised machines as self-hosted GitHub Actions runners named “SHA1HULUD” and triggered them through repository discussions, giving the attacker remote execution. If credential theft failed, it wiped the user's home directory.

Prevention bypassed. Scanners tuned to postinstall and to Node.js. Token rotation after the first wave, which was incomplete across the ecosystem. Backups that excluded developer home directories.

Controls missed.

- Organisation policy restricting who can register self-hosted runners, and an alert on every new runner registration.
- Workflows triggered by discussions or other low-trust events reviewed and restricted.
- Detection of a package install downloading a second runtime (Bun, Deno, Python) from the internet.
- Complete rotation of every token exposed in the September wave, not only the ones named in advisories.
- Endpoint backups that include source code and configuration in home directories.

Detection opportunity. A new self-hosted runner appearing in an organisation's audit log; outbound downloads of a runtime binary during npm install; a sudden rm of a home directory following failed outbound connections.

3.5 Trust Wallet browser extension, 24 December 2025

Ecosystem	Relay	Scale reported
Chrome Web Store	Attributed by the vendor to Shai-Hulud 2.0 exposure	About US\$8.5 million drained from about 2,520 wallet addresses

What happened. A malicious version (2.68) of the Trust Wallet Chrome extension was published to the Chrome Web Store on 24 December. Wallet draining was reported publicly the next day, and users were urged to move to version 2.69.

Entry point. The vendor said the Shai-Hulud 2.0 outbreak was likely responsible, implying that credentials or secrets able to publish the extension were exposed in that wave. The attacker's exfiltration domain was registered on 8 December, more than two weeks before the malicious release.

Payload and behaviour. The tampered extension harvested wallet data and used an analytics library already present in the codebase as a carrier for exfiltration to the look-alike domain.

Prevention bypassed. Browser-store review. Code signing, because the store accepted a release from valid publisher credentials. Rotation after Shai-Hulud 2.0, which evidently did not cover the extension's publishing path.

Controls missed.

- An inventory of every publishing credential the organisation holds, including browser stores and mobile app stores, rotated as one set after an ecosystem worm.
- Store publishing restricted to a protected release pipeline with mandatory human approval.
- Reproducible builds, with each store package compared automatically against the tagged source.
- Brand monitoring for look-alike domains, which would have flagged the 8 December registration.

Detection opportunity. Look-alike domain registration two weeks before release; a store package whose contents differ from the build of its tagged commit; first requests to the new domain from the extension three days before the release, as later analysis showed.

3.6 What the lineage teaches

The lineage shows that a vulnerability in one project's CI becomes everyone's problem once its credentials leak. It shows that two-factor authentication on accounts is irrelevant when publishing runs on stored tokens. And it shows that rotation after a worm must cover every publishing path an organisation owns, including the ones nobody thinks of as software supply chain.

CHAPTER 4

GlassWorm and the IDE extension attack surface (October 2025 – April 2026)

GlassWorm was the first self-propagating worm to spread through code-editor extensions. It returned in at least four waves over six months, and its techniques (invisible code and blockchain-hosted command and control) were copied by later campaigns. It is the clearest case in this book of an attack surface most organisations did not inventory at all.

4.1 The waves

Date	Wave	Scale reported
17–21 Oct 2025	First wave on OpenVSX and the Microsoft VS Code Marketplace	13 OpenVSX extensions and 1 Marketplace extension; about 35,800 downloads; OpenVSX declared it contained on 21 October
Nov 2025	Resurgence targeting GitHub repositories	Not quantified in public reporting
2 Dec 2025	Impersonation wave	24 extensions impersonating popular tools (Flutter, React, Tailwind, Vim, Vue and others) across both marketplaces
Early 2026	Developer-account compromise	Four extensions from one publisher with more than 22,000 downloads, consistent with a leaked publishing token
Mid-Mar 2026	Fourth and largest wave	About 433 components across OpenVSX, the VS Code Marketplace, GitHub and npm (estimate)
Apr 2026	Sleeper clones	More than 70 cloned extensions published, likely awaiting a malicious update

4.2 Incident write-up

What happened. Malicious updates were pushed to legitimate extensions, and later to cloned look-alike extensions. Any developer whose editor auto-updated received the payload.

Entry point. Compromised extension-publisher accounts and leaked publishing tokens. Once running, the worm stole new publisher, npm and GitHub credentials and used them to infect more extensions and packages. Later waves added a social-engineering path: cloned listings that copied icons, names and descriptions to collect installs before any malware was introduced.

Payload and behaviour. The defining technique was invisibility. The malicious code was written in Unicode variation selectors and private-use characters that render as nothing in editors, terminals and code-review diffs. Command and control was resolved from transactions on the Solana blockchain, with a Google Calendar event as a fallback, so there was no domain to take down. The payload harvested npm, OpenVSX, GitHub and Git credentials, targeted 49 cryptocurrency wallet extensions, installed SOCKS proxies that turned developer machines into attacker infrastructure, and installed hidden VNC servers for remote access. Some variants avoided systems with a Russian locale.

Prevention bypassed. Visual code review, because the code could not be seen. Marketplace review, because updates came from legitimate publishers. Domain and IP block-lists, because command and control lived on a public blockchain. Endpoint controls that treat the editor as a trusted application.

Controls missed.

- An allow-list of approved extensions enforced by enterprise policy, rather than free installation from public marketplaces.
- A private or mirrored extension marketplace, with updates staged and reviewed before release to developers.
- Automatic extension updates disabled on high-privilege developer machines, or delayed by a minimum release age.
- Publisher-token hygiene for organisations that publish extensions: short-lived tokens, two-person release, offboarding that revokes namespace tokens.
- Automated detection of invisible and bidirectional Unicode characters in source, packages and extensions.
- No personal cryptocurrency wallets or browser-stored credentials on machines that hold production publishing rights.

Detection opportunity. An editor or extension-host process connecting to blockchain RPC endpoints or calendar APIs; new listening SOCKS or VNC services on a developer workstation; an extension update whose package contains large runs of non-printing Unicode; a burst of extension and package publishes from one developer's tokens.

4.3 Why the editor became a target

An editor extension runs with the developer's full user privileges, has read access to every repository the developer opens, and updates silently. In 2026 the same reasoning extended to AI coding agents. Several later incidents in this book planted files that agents or editors execute when a folder is opened: Nx Console in May, the Microsoft organisation compromise in June and Miasma's Go modules in late June.

The defender's takeaway is to treat the developer workstation as part of the build system. Anything that can run code on it (extensions, agent hooks, workspace tasks, package installs) needs the same intake controls as a CI dependency.

4.4 Related incidents in other chapters

The Trivy precursor (Chapter 5) began with a malicious Trivy extension uploaded to OpenVSX with an ex-employee's unrevoked token. The Nx Console compromise (Chapter 6) put a credential stealer into a popular VS Code extension after a contributor installed a poisoned package. Both are written up in full in their own chapters.

CHAPTER 5

The TeamPCP / Trivy cascade (February – April 2026)

In March 2026 an actor tracked as TeamPCP turned one security scanner's CI into a credential vending machine. Starting from a single stolen bot token, the campaign moved from GitHub Actions to Docker Hub, npm, OpenVSX and PyPI in five days, and kept using leftover credentials for five more weeks. The cascade is the best-documented example of a credential relay and the clearest case against partial rotation.

5.1 Trivy precursor: a stale token and a pwn request, 20–28 February 2026

Ecosystem	Relay	Scale reported
OpenVSX, GitHub Actions	Seed of the TeamPCP cascade	One malicious extension; one stolen bot personal access token

What happened. On 20 February an automated account began scanning public repositories for exploitable `pull_request_target` workflows. On 27 February a malicious Trivy VS Code extension appeared on OpenVSX. On 28 February a pull request against Trivy's CI exfiltrated the personal access token of the project's automation bot.

Entry point. Two independent weaknesses. The extension was published with a former employee's OpenVSX publisher token, a long-lived token tied to the publisher namespace that was not revoked when the person left. The bot token was stolen through a “pwn request”: a `pull_request_target` workflow that executed code from the attacker's fork with access to repository secrets.

Payload and behaviour. The extension abused local AI coding agents to locate and exfiltrate secrets. The pwn request delivered the bot credential to the attacker. Aqua Security began containment on 1 March, but rotation was incomplete.

Prevention bypassed. Offboarding, which removed the employee but not their marketplace token. Fork-PR approval settings, which did not stop a `pull_request_target` workflow from running untrusted code. The project's own scanning, since the weakness was in workflow design, not in a dependency.

Controls missed.

- Offboarding that revokes every registry, marketplace and CI token an individual created, driven by a token inventory rather than by account deletion.
- A ban on `pull_request_target` workflows that check out or execute pull-request code, enforced by a workflow linter in CI.
- Bot and service-account tokens scoped to the minimum repositories and permissions, and short-lived.
- Atomic rotation of every credential the bot could reach after the first sign of compromise.

Detection opportunity. An unfamiliar account opening pull requests across many repositories with workflow-targeting changes; a publish to OpenVSX by a namespace token not used in months; the bot token used from an unfamiliar network immediately after a fork pull request ran.

5.2 Trivy / Aqua Security main compromise, 19–20 March 2026

Ecosystem	Relay	Scale reported
GitHub Actions, GitHub Releases, Docker Hub, GHCR, ECR	TeamPCP	75 of 76 trivy-action tags and all 7 setup-trivy tags repointed; action compromise window about 12 hours

What happened. At 17:43 UTC on 19 March the attacker force-pushed 75 of 76 version tags in the trivy-action repository and all seven tags in setup-trivy to malicious commits, using the compromised bot account. At 18:22 UTC a backdoored Trivy release, v0.69.4, was published to GitHub Releases, Docker Hub, GHCR and ECR. Any pipeline that referenced Trivy by tag ran the attacker's code. The release artefacts were public for about three hours and the action tags for about twelve.

Entry point. The bot credential stolen in February. Rotation after the 1 March containment was incomplete, or the attacker obtained fresh tokens; either way the bot still had tag-write and release rights.

Payload and behaviour. The payload read the GitHub Actions Runner.Worker process memory to find values marked as secrets, bypassing log masking entirely. It scanned the runner's local /24 network, encrypted what it found and sent it to a typosquatted command-and-control domain. Some Python stages were hidden inside audio file frames.

Prevention bypassed. Pinning by version tag, which most consumers believed was safe. Log masking of secrets. Rotation after the February breach. The scanner's own reputation: consumers ran it with broad permissions precisely because it was a security tool.

Controls missed.

- Consumers pinning third-party actions to full commit SHAs, and organisation policy enforcing SHA pinning.
- Repository rulesets that block force-pushes and updates to release tags, and immutable releases.
- Separation of release rights from the automation bot, with human approval on a protected environment for any tag or release.
- Scanner jobs given read-only tokens and no access to deployment or publishing secrets.
- Egress restrictions on CI runners, so an unknown domain cannot receive exfiltrated data.

Detection opportunity. Dozens of tag force-pushes in minutes in the audit log; a release published by a bot outside the normal release workflow; runner processes reading another process's memory; CI egress to a domain first seen that day.

5.3 Spam flood to bury the disclosure, 20 March 2026

What happened. As the community began reporting the compromise, the disclosure thread was flooded with spam from a mix of compromised developer accounts and purchased dormant accounts.

Why it matters. The flood slowed defenders' ability to find indicators and instructions. It is the first clear example in this book of an attacker targeting the incident response process itself.

Controls missed. A pre-agreed out-of-band channel for maintainers and major consumers; an advisory published through a security-advisory mechanism rather than an open issue thread; the ability to lock and moderate a thread quickly.

5.4 CanisterWorm, 20–21 March 2026

Ecosystem	Relay	Scale reported
npm	TeamPCP	47 or more packages in the first day

What happened. npm publish tokens exfiltrated from CI pipelines during the Trivy wave seeded a self-propagating npm worm the next day.

Entry point. Long-lived npm publish tokens stored as CI secrets in pipelines that ran the compromised Trivy action.

Payload and behaviour. During install or CI, the worm ran npm whoami to identify the user, listed every package that user maintained, searched the environment for more npm tokens, bumped the patch version, injected itself and republished. It repeated from every newly infected consumer or maintainer, using blockchain-hosted infrastructure for command and control.

Prevention bypassed. The organisations that lost these tokens were not compromised directly; they had run a trusted security scanner. Lockfiles in consumers that pulled fresh patch versions.

Controls missed. Trusted publishing instead of stored npm tokens; publish secrets never present in jobs that run third-party tools; minimum release age for dependency updates; install scripts disabled for dependencies.

Detection opportunity. A wave of patch-version publishes from tokens belonging to many unrelated maintainers within hours; npm whoami and npm publish executed from inside a package's install script.

5.5 Docker Hub persistence and repository defacement, 22–23 March 2026

Ecosystem	Relay	Scale reported
Docker Hub, GitHub	TeamPCP	Two malicious images live about 9.5 hours; 44 internal repositories renamed and made public

What happened. Malicious aquasec/trivy images 0.69.5 and 0.69.6 were pushed to Docker Hub and were live for about 9.5 hours. In a scripted two-minute burst, the attacker renamed 44 repositories in Aqua's internal GitHub organisation with a "tpcp-docs-" prefix and made the internal source code public.

Entry point. A Docker Hub credential exfiltrated on 19 March and not yet revoked, plus GitHub access sufficient to change repository visibility across the organisation.

Payload and behaviour. Any CI job or developer that pulled and ran the new tags executed a stealer that collected CI secrets, cloud credentials and tokens.

Prevention bypassed. Rotation of GitHub credentials after the main compromise, which did not include the Docker Hub credential. Consumers' trust in the official vendor namespace on Docker Hub.

Controls missed. Consumers pulling images by digest and verifying signatures; registry push restricted to the release pipeline with signed images only; organisation policy preventing members from making repositories public, with alerts on visibility changes; rotation that covered every registry, not only the source host.

Detection opportunity. New image tags pushed outside the release pipeline and without a signature; a burst of repository rename and visibility events in the audit log.

5.6 Checkmarx KICS GitHub Action and OpenVSX extension, 23 March 2026

Ecosystem	Relay	Scale reported
GitHub Actions, OpenVSX	TeamPCP	Action tags redirected; one malicious extension

What happened. A Checkmarx release service account was used to redirect the KICS action's tags to malicious commits, and a malicious OpenVSX extension was published through a compromised account.

Entry point. The service account's credentials were exfiltrated during the 19 March Trivy wave, either from a CI pipeline or from a developer workstation.

Payload and behaviour. A malicious setup.sh scraped CI/CD secrets from runners that used the action and exfiltrated them to attacker infrastructure.

Controls missed. The same as Trivy: SHA pinning by consumers, tag protection by the publisher, release rights separated from long-lived service accounts. In addition, a security vendor's own pipelines ran another vendor's compromised scanner with access to release credentials.

Detection opportunity. Tag updates by a service account outside a release; a vendor's service account active from a new location within days of an industry-wide advisory.

5.7 LiteLLM on PyPI, 23–24 March 2026

Ecosystem	Relay	Scale reported
GitHub Actions, PyPI	TeamPCP	Malicious litellm 1.82.7 and 1.82.8

What happened. Using a compromised personal access token from the maintainer organisation, the attacker added workflows that dumped every repository secret. One of those secrets was a long-lived PyPI publishing token, which was used to publish malicious litellm versions 1.82.7 and 1.82.8 on 24 March.

Entry point. A PAT stolen in the Trivy wave, with permission to add workflow files.

Payload and behaviour. At least one malicious version used a .pth file, which Python executes every time the interpreter starts, not only when litellm is imported. A spam flood was again observed during disclosure.

Prevention bypassed. Repository secrets were assumed to be safe unless a workflow printed them. Install-hook scanners did not watch .pth files.

Controls missed. PyPI trusted publishing (the project adopted it on 2 April); PATs that cannot modify workflow files; CODEOWNERS and required review for changes under .github/workflows; secrets bound to protected environments so a new workflow cannot read them.

Detection opportunity. A new workflow file added directly to a branch and run once; a workflow that references every secret; a PyPI release from a token rather than the project's usual publishing path.

5.8 Telnyx on PyPI, 27 March 2026

Ecosystem	Relay	Scale reported
PyPI	TeamPCP (probable)	Malicious Telnyx package versions

What happened. A PyPI API token was used to upload a malicious package directly to PyPI, with no change to the source repository.

Entry point. The probable chain: a Telnix CI job ran the compromised Trivy, the stealer collected the PyPI token, and the attacker used it eight days later.

Payload and behaviour. The payload ran at import time, avoiding scanners that only watch install scripts. It downloaded a .wav audio file, decoded a credential-collector script hidden inside it, ran the script in memory, encrypted the results and posted them to the attacker's server.

Controls missed. Trusted publishing; publish credentials kept out of any job that runs third-party tooling; consumer-side checks that a release has provenance tied to the project's repository.

Detection opportunity. A PyPI release with no matching tag or commit in the source repository; Python processes downloading audio files at import.

5.9 Response milestone: trusted publishing adopted, 2 April 2026

LiteLLM and Telnix replaced long-lived PyPI tokens with trusted publishing, a short-lived, identity-bound exchange between the CI provider and the registry. This removed the reusable credential the attacker had relied on. It is the single most effective control change recorded in the cascade, and it came after the damage.

5.10 fairwords and CanisterSprawl, 8 April and 22–23 April 2026

Ecosystem	Relay	Scale reported
npm, PyPI	TeamPCP (probable)	fairwords; pgserve, automagik, openwebconcept and others

What happened. Malicious npm versions of fairwords appeared on 8 April, and of pgserve, automagik, openwebconcept and related packages on 22–23 April.

Entry point. Not publicly disclosed. The timing and payload suggest tokens stolen in the March wave and never rotated, 20 to 34 days earlier.

Payload and behaviour. A postinstall hook harvested environment variables matched by keywords such as token, secret, aws, npm, github, openai and anthropic, along with cloud credentials, Kubernetes and Docker configuration, shell history, cryptocurrency wallets and browser-stored passwords. It encrypted the data and sent it to a canister on the Internet Computer blockchain. It then listed every package any discovered npm token could publish, bumped the patch version and republished itself. If it found a PyPI token, it built and uploaded a package carrying a malicious .pth file, crossing ecosystems on its own.

Controls missed. Rotation of every token exposed in March, including those belonging to organisations that only consumed Trivy; developer machines free of browser-stored passwords and long-lived tokens; detection of AI-provider API keys as high-value secrets.

5.11 Checkmarx KICS Docker images and Bitwarden CLI, 22 April 2026

Ecosystem	Relay	Scale reported
Docker Hub, npm	TeamPCP	Malicious KICS images; one malicious Bitwarden CLI npm release

What happened. Malicious KICS Docker images and a malicious Bitwarden CLI npm package were published on the same day, five weeks after the original theft.

Entry point. For KICS, a Docker Hub publisher token from the March wave that had not been cleaned up. For Bitwarden CLI, a compromised publishing path that public reporting links to the same credential theft.

Payload and behaviour. KICS scans infrastructure-as-code such as Terraform, Kubernetes manifests and Dockerfiles, and its reports routinely contain hard-coded secrets it finds; normal output redacts them, but the malicious image sent the attacker the unredacted results. The Bitwarden CLI package used a preinstall hook to download the Bun runtime and run an obfuscated stealer that harvested GitHub and npm tokens, AWS, GCP and Azure credentials, SSH keys, environment variables and CI secrets, exfiltrating through repositories branded “Shai-Hulud: The Third Coming.” Bitwarden confirmed that no vault data or end-user password data was accessed; the damage was to the machines and runners that installed the CLI.

Prevention bypassed. A clean-up that was declared complete in March. Users' trust in a password manager vendor's official CLI.

Controls missed. A credential inventory that would have shown the surviving Docker Hub token; signed images verified on pull; a 90-day watch on every credential retired after an incident; minimum release age for security-tool updates in CI.

5.12 What the cascade teaches

The cascade shows that consuming a compromised tool is enough to become a victim, and that every organisation that ran Trivy by tag on 19 March should have treated its CI secrets as stolen. It shows that rotation scoped to the obvious credential leaves the attacker a door for weeks. And it shows that the decisive fix, trusted publishing, was available throughout and adopted only after the breach.

CHAPTER 6

Mini Shai-Hulud, TanStack, Miasma and ChainDrop (April – September 2026)

The third relay was faster and more technically advanced than the first two. It began with a CI setting that leaked secrets to forked pull requests, escalated through a cache-poisoning attack that stole an OIDC token from runner memory, and by August produced a worm that poisoned 444 packages in about four hours. It also reached large enterprises through their employees' and contributors' personal tokens. Public reporting links this relay to the TeamPCP actor.

6.1 Mini Shai-Hulud: SAP packages, 29–30 April 2026

Ecosystem	Relay	Scale reported
CircleCI, npm, PyPI	Seed of the Mini Shai-Hulud wave	Release secrets including npm, PyPI and iOS signing certificates

What happened. A draft pull request was opened from a fork, and CircleCI ran the build for that fork pull request with the project's secrets available.

Entry point. SAP's release secrets were stored in CircleCI project settings, and the setting “Pass secrets to builds from forked pull requests” was effectively enabled. The pull request changed the test command to run a stealer.

Payload and behaviour. The stolen secrets included npm and PyPI publishing tokens and an iOS signing certificate. Malicious npm and PyPI packages followed, carrying obfuscated JavaScript, hidden import-time execution in the Python packages, downloads of the Bun runtime, and fallback exfiltration through GitHub accounts with Dune-themed names.

Prevention bypassed. The assumption that a draft pull request is inert. Repository-level review, since the change was never merged.

Controls missed. Secrets never exposed to forked pull-request builds on any CI platform; release secrets held in restricted contexts or environments that only protected branches can reach; trusted publishing; signing certificates in a hardware-backed signing service rather than as a CI secret.

Detection opportunity. A fork build that modified the test command and made outbound connections; a CI job reading secrets it does not normally use.

6.2 Cemu release assets swapped, 7 May 2026

Ecosystem	Relay	Scale reported
GitHub Releases	Mini Shai-Hulud (likely)	Linux Applmage and Ubuntu artefacts replaced

What happened. The Linux release binaries of the Cemu emulator were replaced through the GitHub Releases API. There was no new commit, no tag change, no workflow edit and no CI run.

Entry point. A long-standing contributor's account, most likely compromised in the concurrent Mini Shai-Hulud harvest, used with a personal token that could edit release assets.

Payload and behaviour. A Python application that ran only on Linux, only outside Russian locales and only on machines with more than four CPUs. It contained destructive logic: a geofence for Israel and Iran with a one-in-six chance of wiping the disk.

Prevention bypassed. Every control that watches commits, tags and CI. Release assets were mutable and could be changed by any account with write access.

Controls missed. Immutable releases; release assets published only by the release workflow with signed checksums and attestations; contributor tokens without release-edit rights; consumers verifying signatures or checksums before running a binary.

Detection opportunity. A release-asset replacement event in the audit log with no associated workflow run; a published checksum that no longer matches the asset.

6.3 TanStack npm compromise, 11 May 2026

Ecosystem	Relay	Scale reported
GitHub Actions, npm	Mini Shai-Hulud	84 versions across 42 packages published in about 6 minutes

What happened. An attacker opened a pull request from a fork and force-pushed to it repeatedly, poisoning the project's pnpm dependency cache. A trusted release workflow later restored and used the poisoned cache. That job had the id-token: write permission, so the malware read the job's OIDC token out of runner memory and used it to publish 84 malicious versions across 42 packages in about six minutes.

Entry point. A pull_request_target workflow that checked out or executed untrusted pull-request code and wrote to a cache shared with privileged workflows.

Payload and behaviour. The payload read /proc/vmem to extract OIDC tokens and secrets from build processes. Public reporting of the wave describes the same memory-dumping logic for Linux, macOS and Windows, cloud-access validation, persistence, and fallback exfiltration by committing JSON files to attacker-created repositories.

Prevention bypassed. Trusted publishing itself. The project used OIDC rather than a stored token, but the attacker stole the short-lived token from inside the trusted job. Provenance, because the publish came from the real workflow.

Controls missed.

- Caches isolated between untrusted (pull request) and trusted (release) workflows, or no cache restore in release jobs.
- Release jobs run on fresh, ephemeral runners with no restored state from less-trusted events.
- id-token: write granted only to a minimal publish job in a protected environment, not to jobs that build or test.
- Trusted-publisher configuration pinned to a specific workflow file, branch and environment.
- Egress filtering on release runners.

Detection opportunity. Many force-pushes to a fork pull request in a short period; a release job whose cache key was written by a pull-request run; dozens of package versions published within minutes.

6.4 UiPath and Mistral packages, 11 May 2026

Ecosystem	Relay	Scale reported
npm, PyPI	Mini Shai-Hulud	UiPath npm packages republished; Mistral npm and PyPI packages

What happened. Tokens stolen in the same wave were used the same day to republish UiPath npm packages and to publish malicious Mistral npm and PyPI packages.

Payload and behaviour. The npm payload was a credential stealer and self-propagating worm, though a bug made it non-functional. The PyPI payload worked: it downloaded a Python zip application with TLS certificate verification disabled, ran it, harvested credentials associated with 1Password and Bitwarden, and exfiltrated them, with GitHub as a fallback channel.

Controls missed. Trusted publishing for both ecosystems; developer endpoints that do not keep password-manager sessions unlocked for command-line access by any process; detection of Python processes disabling TLS verification.

6.5 Nx Console VS Code extension, 18 May 2026

Ecosystem	Relay	Scale reported
VS Code Marketplace, OpenVSX	Mini Shai-Hulud	One malicious release of a widely used extension

What happened. A malicious Nx Console extension release was published with a stolen token.

Entry point. An Nx contributor installed a poisoned TanStack package on 11 May; the payload stole a GitHub token that could publish the extension.

Payload and behaviour. The extension targeted GitHub, npm, AWS, HashiCorp Vault, Kubernetes and 1Password credentials and installed a persistence mechanism disguised as a monitoring process.

Prevention bypassed. The Nx project had already lived through the August 2025 compromise. The new entry point was a contributor's personal machine, outside the project's controls.

Controls missed. Extension publishing restricted to a release pipeline with approval, not to contributors' tokens; contributor tokens fine-grained and short-lived; organisations consuming extensions with an allow-list and delayed updates.

6.6 Actions-cool GitHub Actions, 18–19 May 2026

Ecosystem	Relay	Scale reported
GitHub Actions	Mini Shai-Hulud (possible)	Every tag force-moved to imposter commits

What happened. An attacker with push and tag-write access force-moved every tag in the Actions-cool repositories to imposter commits.

Payload and behaviour. The malicious workflows downloaded a helper, ran JavaScript that extracted secrets from runner memory, and exfiltrated the runner's GitHub token.

Controls missed. SHA pinning by consumers; tag protection by the publisher; organisation policy allowing only approved actions.

6.7 AntV npm family, 18–19 May 2026

Ecosystem	Relay	Scale reported
GitHub, npm	Mini Shai-Hulud (possible)	639 malicious versions across 323 packages

What happened. Imposter commits were planted and a maintainer's npm account was compromised. The attacker published 639 malicious versions across 323 packages.

Payload and behaviour. An obfuscated Bun bundle, triggered by a preinstall hook, targeted AWS environment variables, ECS metadata, GCP service accounts, Azure credentials, Kubernetes tokens, GitHub PATs, npm tokens, Vault, Stripe and Slack secrets and database connection strings, and exfiltrated them to attacker-controlled GitHub repositories.

Controls missed. Package-scoped publish tokens, so one account cannot publish hundreds of packages; publish rate alerts; consumer cooldowns; detection of cloud metadata endpoint access from install scripts.

6.8 Microsoft DurableTask on PyPI, 19 May 2026

Ecosystem	Relay	Scale reported
PyPI	Mini Shai-Hulud	Three malicious versions

What happened. Three malicious DurableTask versions were published to PyPI with a token stolen during the AntV campaign the day before.

Payload and behaviour. The payload targeted AWS Systems Manager and Kubernetes for lateral movement, moving beyond credential theft to active use of cloud control planes.

Controls missed. Trusted publishing; separation between the machines that hold publishing tokens and the machines that install third-party packages; cloud detections for SSM and Kubernetes API use from build or developer identities.

6.9 Miasma: Red Hat npm packages, 1–8 June 2026

Ecosystem	Relay	Scale reported
GitHub Actions, npm	Mini Shai-Hulud / Miasma	90 versions across 32 packages, with valid SLSA provenance

What happened. The attacker held an OAuth token with write access to specific Red Hat Cloud Services repositories, through a compromised employee GitHub account. They pushed a new branch containing a workflow file with the same name as the project's legitimate one. GitHub ran it on push and issued an OIDC token that truthfully identified the repository and the workflow file name. npm's trusted-publisher configuration matched and allowed the publish.

Entry point. A personal access or OAuth token scoped to the repositories the employee worked on. The most likely source is the employee installing a package compromised earlier in the wave.

Payload and behaviour. The payload downloaded Bun, harvested developer and CI credentials and spread to several more npm accounts. Later versions moved execution into binding.gyp, which can trigger shell evaluation through the native-module build step during install.

Prevention bypassed. Trusted publishing and SLSA provenance. Both worked as designed; they attested a workflow the attacker controlled.

Controls missed.

- Trusted-publisher configuration restricted to a protected branch or tag and a protected environment, not only to a workflow file name.
- Branch protection and rulesets that stop new branches from running privileged workflows, and CODEOWNERS review for any workflow file.
- Consumers verifying that provenance names an expected ref (a release tag on the default branch), not just an expected repository.
- Employee OAuth and personal tokens governed by organisation policy, with expiry and approval.

Detection opportunity. A publish whose provenance names a non-default branch; a workflow file created on a new branch and run immediately; dozens of versions across a package scope in a short period.

6.10 Microsoft and Azure GitHub organisations, 5 June 2026

Ecosystem	Relay	Scale reported
GitHub repositories	Mini Shai-Hulud	42 repositories and 236 branches modified in 46 minutes

What happened. Using one compromised GitHub account that had write access to repositories in several Microsoft organisations, the attacker injected malicious code into 42 repositories and 236 branches in 46 minutes. The code included a `.github/setup.js` and auto-execution hooks for Claude Code, Gemini CLI, Cursor and VS Code. Public reporting says Microsoft disabled the Azure Functions Action and dozens of other repositories while it responded.

Entry point. A contributor's GitHub token stolen about a month earlier, in the TanStack-era harvest.

Payload and behaviour. Nothing was shipped to a package registry. The goal was to infect the next developer or AI agent to open the repository.

Prevention bypassed. Branch protection on default branches, since the code went to many branches. Package-registry monitoring, since there was no package.

Controls missed. Rulesets protecting all branches that matter, not only the default; contributor access reviewed and time-bound; agent and editor settings files treated as code requiring review; developer tooling configured not to run repository-supplied hooks without trust.

Detection opportunity. One account pushing to hundreds of branches across organisations within an hour; new files in agent or editor configuration paths.

6.11 Hades: 37 PyPI wheels, 7–8 June 2026

Ecosystem	Relay	Scale reported
PyPI	Miasma / TeamPCP-linked	37 wheels across 19 packages, extending into bioinformatics packages

What happened. The campaign compromised 37 wheels across 19 PyPI packages, using `.pth` startup execution.

Payload and behaviour. The payload reportedly included fake jailbreak-style text intended to derail AI-assisted malware analysis.

Controls missed. Trusted publishing; consumer controls that treat `.pth` files in third-party wheels as executable; analysis pipelines that do not let sample content instruct an AI analyst.

6.12 Miasma wave 2, 24–26 June 2026

Ecosystem	Relay	Scale reported
GitHub Actions, npm, Go modules	Miasma	Two GitHub Actions; a burst of npm packages; Go modules

What happened. Late Miasma activity compromised two GitHub Actions, followed by a rapid burst of LeoPlatform npm packages and more releases through another compromised account. It also produced Go modules with payloads in `.claude/index.js`, designed to run when developers open the repository in an IDE or AI tool rather than through the Go toolchain.

Controls missed. SHA pinning; agent configuration review; developer AI tools configured to require explicit trust before running repository-provided hooks or scripts.

6.13 ChainDrop worm (keyv family), 4 August 2026

Ecosystem	Relay	Scale reported
GitHub, npm	Mini Shai-Hulud variant linked to TeamPCP	444+ packages and about 2,200 versions in about 4 hours

What happened. At 09:02 UTC on 4 August an unsigned commit added a malicious preinstall hook and payload files to the keyv repository. The attacker had taken over the maintainer's GitHub account, pushed directly to the main branch and immediately cut a release, so the poisoned versions were published to npm with valid provenance signed by GitHub Actions. The worm then spread through stolen npm tokens to more than 444 packages and about 2,200 versions in roughly four hours, at its peak about one infected package per second. The keyv family alone accounts for well over a hundred million weekly downloads. Singapore's Cyber Security Agency issued a national advisory on 6 August.

Entry point. The maintainer's GitHub account and the absence of review between a push to main and a release.

Payload and behaviour. The payload harvested npm tokens from every `.npmrc` on the machine, AWS, GCP and Azure keys, Vault tokens, Kubernetes service-account credentials and GitHub Actions OIDC tokens. It resolved command-and-control addresses from an Ethereum smart contract, so the attacker could rotate domains with one blockchain transaction. It planted payloads in AI-agent configuration files. If a stolen token was revoked, a daemon wiped the victim's home directory.

Prevention bypassed. Provenance and trusted publishing. Domain block-lists. The standard response step of revoking a stolen token, which triggered the wiper.

Controls missed.

- Signed commits required and direct pushes to the default branch blocked, even for maintainers.
- Release workflows that require a second maintainer's approval in a protected environment.
- Consumers enforcing a minimum release age and running installs with scripts disabled.
- A response plan that isolates or snapshots an infected machine before revoking tokens from it.

Detection opportunity. An unsigned commit to main followed within minutes by a release; Ethereum RPC calls from CI runners or developer machines; a sudden spike of publishes across many unrelated maintainers.

6.14 CrowdSec downstream breach, disclosed 17 September 2026

What happened. CrowdSec disclosed a compromise downstream of the TanStack wave, in which a stolen OAuth token was used to clone private repositories. It became public only after a later leak.

Why it matters. It shows a stolen token from May being exploited, or its theft discovered, four months later. Source code, and any secrets committed in it, should be treated as exposed whenever a token with read access is stolen.

Controls missed. OAuth app and token inventory with expiry; alerts on bulk clone activity; secrets never committed to private repositories.

6.15 What the wave teaches

The wave shows that short-lived credentials are not enough when the attacker can run code inside the job that holds them. It shows that provenance answers “which workflow built this?” and not “should this have been built?”. And it shows that an enterprise's exposure includes every personal token its employees and contributors hold, on machines it does not manage.

CHAPTER 7

Independent actors and the AI ecosystem (June – September 2026)

By mid-2026 the relay playbook had spread beyond the actors who built it. The two incidents in this chapter were run by different operators, against different targets, with different tooling, and they used the same weaknesses: an access grant nobody revoked, and a release job that trusted whatever ran before it. The second also shows attackers following developers into AI tooling.

7.1 Mastra and easy-day-js, 17 June 2026

Ecosystem	Relay	Scale reported
npm	Independent (one public analysis attributes it to a state-linked actor tracked as Sapphire Sleet)	More than 140 packages

What happened. Malicious versions of more than 140 packages in the Mastra AI-framework scope were published. The library code in each release was byte-for-byte identical to the legitimate version. The only change was one added dependency.

Entry point. A former contributor's npm publisher account, dormant for about 16 months, still had publish access to the scope. Its email address had been changed to a privacy-mail address before the attack. The malicious versions were published from a personal token with no provenance attestations, unlike the project's legitimate releases.

Payload and behaviour. The added dependency, easy-day-js, was a clone of the popular dayjs date library. Its postinstall hook dropped and ran a multi-platform remote-access trojan that stole cryptocurrency wallets, browser data and cloud credentials.

Prevention bypassed. Code review and diff-based scanning of the package's own code, which had not changed. Typosquat detection, since the scope was genuine. Provenance, which was simply absent and not required.

Controls missed.

- Periodic access reviews of registry scopes, removing any member inactive for a set period (for example, 90 days).
- An alert to scope owners whenever a member's email or two-factor settings change.
- Scope-level policy requiring that releases come only through trusted publishing with provenance.
- Consumer policy rejecting a release that lacks provenance when previous releases had it.
- Consumer review of new transitive dependencies introduced by an update, not only version changes.

Detection opportunity. A publish from an account that had not published in over a year; a release whose only change is a new dependency on a package less than days old; dist.attestations missing on a release of a package that normally carries them.

7.2 MemTensor / MemoryOS and the “sckit” worm, 23 September 2026

Ecosystem	Relay	Scale reported
GitHub Actions, npm, PyPI	Independent (operator self-identifies only by a module name)	npm plugin versions 0.1.21, 0.1.23 and 0.1.25; PyPI MemoryOS 2.0.34; exposure about 3.5 hours

What happened. Between about 02:23 and 05:55 UTC on 23 September, backdoored versions of two packages from MemTensor, maker of the MemOS memory framework for AI agents, were the default install on npm and PyPI. The npm package was a memory plugin for an AI agent harness. A researcher raised the alarm on GitHub at 04:17 UTC.

Entry point. The npm plugin was published by a GitHub Actions release workflow that read its publish token from a repository secret. Operating through a GitHub account in the organisation, the attacker pushed a short-lived branch named like a release branch, made a three-line change to a validation script that runs earlier in the same job, then deleted the branch. The change used the BASH_ENV variable so that every later shell step in the job ran attacker code, capturing the token. The PyPI compromise used the same technique through a different entry point.

Payload and behaviour. Both packages carried a cross-platform Go binary, called sckit by its author and built under a module path of supplychain.local/campaign, for Windows, Linux and macOS on ARM and x86. It runs when the package is imported or invoked, not at install time. It searches for JWTs, AWS keys, GitHub and GitLab tokens, npm and PyPI tokens, Hugging Face tokens, Vault tokens, Slack, Stripe and SendGrid keys. It carries the logic to republish itself through npm and PyPI and a GitHub Actions template that triggers on every push to a compromised repository.

Prevention bypassed. Install-script scanning, because the code runs at import. Repository review, because the branch existed for minutes and never reached a pull request. Secret masking, because the secret was captured inside the job.

Controls missed.

- Publish secrets bound to a protected environment that only the default branch or a protected tag can use, so a new branch cannot reach them.
- Rulesets restricting who can create branches matching release patterns, and blocking branch deletion that would hide evidence.
- Release jobs split so the publish step runs alone, after validation, in a fresh job that does not inherit environment variables from earlier steps.
- Trusted publishing restricted to a specific workflow, ref and environment.
- Consumers treating CI runners that only ran tests with an affected version as compromised.

Detection opportunity. A branch created, run and deleted within minutes; a workflow writing to BASH_ENV or GITHUB_ENV in an unusual step; a release built from a non-default ref; a Go binary appearing inside a JavaScript or Python package.

7.3 Other activity in the same period

Public reporting describes further 2026 campaigns that reused the same playbook and are not written up in full here. They include a cross-registry campaign that planted 34 malicious packages across npm, PyPI and crates.io using a postinstall hook, import-time code and a Rust build.rs script respectively; more than 700 poisoned versions of a PHP localisation package distributed through rewritten Git tags; a swarm of automated agents that

published more than 3,000 RubyGems packages between May and July after the registry issued publish keys before email verification; and several June–July npm campaigns by other actors. Readers should treat these as evidence that the techniques in this book are now commodity, not the work of a single group.

7.4 What these incidents teach

They show that access left behind is as dangerous as access stolen: a 16-month-old membership was enough to poison 140 packages. They show that any job that holds a publish secret must be treated as untrusted from its first step if anything in it can be influenced by a branch push. And they show attackers moving into AI-agent ecosystems, where packages are new, maintainers are small teams and credentials for AI providers are especially valuable.

PART III

Analysis

CHAPTER 8

The prevention methods that were bypassed

Almost every incident in Part II happened to an organisation that had security controls in place. The controls were real; the assumptions behind them were wrong. This chapter lists eighteen widely used prevention methods, what defenders assumed each one guaranteed, and how attackers walked around it. The final column points to the control in Part IV that closes the gap; control IDs are defined in Chapters 11 to 13 and listed in Chapter 15.

8.1 The eighteen bypassed defences

#	Prevention method	What defenders assumed	How it was bypassed	Incidents	Close the gap with
1	One-time-password two-factor authentication on accounts	A stolen password cannot publish	The code was phished in real time; and automation tokens never prompt for 2FA at all	chalk/debug, Shai-Hulud, most token-based publishes	ID-01, PUB-01, SEC-02
2	Code review of pull requests	Malicious code must pass a reviewer	The exploit was a PR title; the code was invisible Unicode; or no PR existed (branch push, release-asset swap, dependency-only change)	Nx, GlassWorm, MemTensor, Cemmu, Mastra	CI-02, SCM-05, SCM-01, ART-03, DEP-05
3	Branch protection on the default branch	Only reviewed code reaches production	Privileged workflows ran from new branches; 236 branches were modified; a maintainer pushed straight to main	Miasma, Microsoft orgs, ChainDrop, MemTensor	SCM-01, SEC-04, SCM-04
4	Pinning actions and images by version tag	A tag points to the code that was reviewed	Tags were force-moved to malicious commits	Trivy, Checkmarx KICS, Actions-cool	CI-05, SCM-02, ART-01
5	Lockfiles	Builds only use known versions	Fresh installs and automated updates resolved new versions; an update added a new transitive dependency	Shai-Hulud, CanisterWorm, AntV, Mastra	DEP-01, DEP-02, DEP-05
6	Package reputation, allow-lists and typosquat detection	Well-known names from known maintainers are safe	Every malicious version came from the genuine package and account	All registry incidents	DEP-02, DEP-07, PUB-05
7	Registry malware scanning	The registry removes malware before it spreads	Worms published faster than takedown: 84 versions in 6 minutes; about one package per second	TanStack, ChainDrop	DEP-02, DEP-04
8	Scanning or disabling install scripts	Malware runs at install time	Code ran at import, at interpreter start (.pth), in native builds (binding.gyp) or when an editor opened a folder	Telnyx, LiteLLM, Hades, Miasma, MemTensor, Microsoft orgs	DEP-08, END-03, DET-03
9	Masking secrets in CI logs	Secrets are safe if not printed	Payloads read secrets straight from the runner's process memory	Trivy, TanStack, Actions-cool	CI-09, CI-10, CI-08
10	Trusted publishing (OIDC)	No stored token means nothing to steal	The short-lived token was stolen inside the trusted job, or minted for an attacker's workflow	TanStack, Miasma	CI-06, CI-09, SEC-02 (bound to ref and environment)
11	Provenance and SLSA attestations	A signed provenance statement means a release is genuine	The attested workflow was attacker-controlled; elsewhere provenance was absent and nobody checked	Miasma, ChainDrop, Mastra	DEP-07, PUB-02, PUB-03

#	Prevention method	What defenders assumed	How it was bypassed	Incidents	Close the gap with
12	Credential rotation after an incident	Rotation ends the attacker's access	Rotation covered the obvious credential and missed others for up to five weeks	Trivy, KICS Docker, Bitwarden CLI, Shai-Hulud 2.0	SEC-01, SEC-07, DET-07
13	Offboarding through the identity provider	Removing the account removes the access	Marketplace and registry tokens and scope memberships outlived the person by months	Trivy precursor, Mastra	TPR-01, ID-03
14	Approval for workflows from forks	Outside contributors cannot reach secrets	pull_request_target, fork-PR secret settings and shared caches gave untrusted code privileged context	Nx, Trivy precursor, SAP, TanStack	CI-01, CI-04, CI-06
15	Domain and IP block-lists	Blocking the C2 domain stops exfiltration	Command and control lived on Solana, the Internet Computer and Ethereum; exfiltration went to GitHub	GlassWorm, CanisterWorm, CanisterSprawl, ChainDrop, Shai-Hulud	CI-08, DET-09
16	Endpoint security tuned for servers and Node.js	Malware looks like known binaries on known hosts	Payloads downloaded Bun, shipped Go binaries and ran on unmanaged developer laptops	Shai-Hulud 2.0, Bitwarden CLI, AntV, MemTensor	END-01, DET-04
17	Revoking a stolen token as the first response	Revocation is always safe	Revocation triggered a home-directory wiper	Shai-Hulud 2.0, ChainDrop	RSP-02, END-06
18	AI-assisted malware analysis	An AI analyst gives a neutral reading	Samples carried prompt-injection text aimed at the analyst	Hades	RSP-06

8.2 Three patterns behind the bypasses

Controls guarded the front door while attackers used the service entrance. Review, branch protection and account two-factor authentication all protect the path a human developer takes. Attackers used the machine paths: tokens, workflows, tags, caches and release APIs.

Controls verified identity, not intent. Trusted publishing and provenance prove which workflow produced an artefact. They do not prove that the workflow should have run. The fix is to bind identity to the protected ref, environment and approver, so that only the intended path can produce a valid identity.

Controls assumed a single moment of execution. Install-script scanning assumes malware runs at install. In this period code ran at import, at interpreter start, at native build, at editor open and at agent start. Intake controls must treat every one of those as execution.

8.3 What still worked

Some defences held. Community monitoring found the chalk/debug and MemTensor compromises within hours. Trusted publishing ended the reuse of stolen PyPI tokens for the projects that adopted it in April. A seven-day minimum release age on dependency updates would have skipped every malicious version in this book that was removed within a week, which is most of them. Chapter 11 builds on these.

CHAPTER 9

The security controls that were missed

Two controls would have changed the outcome of more than a third of the year's incidents. Replacing long-lived publish tokens with bound trusted publishing was missing in 12 of 33 incidents, and alerting on source-control audit events was missing in 11. Four dependency-intake and credential controls follow closely. This chapter maps every incident to the controls it defeated, so a reader can test their own programme against the record.

9.1 How the mapping was built

Each incident in Part II was reviewed against the 73 controls defined in Part IV. A control is counted as missed when its presence would have prevented the incident, contained it within hours, or made it detectable at the first step. Milestones and the spam flood's response gap are excluded from the count except where noted. The mapping is a judgement from public reporting, not an audit of any organisation; its purpose is to rank controls, not to assign blame.

9.2 The controls missed most often

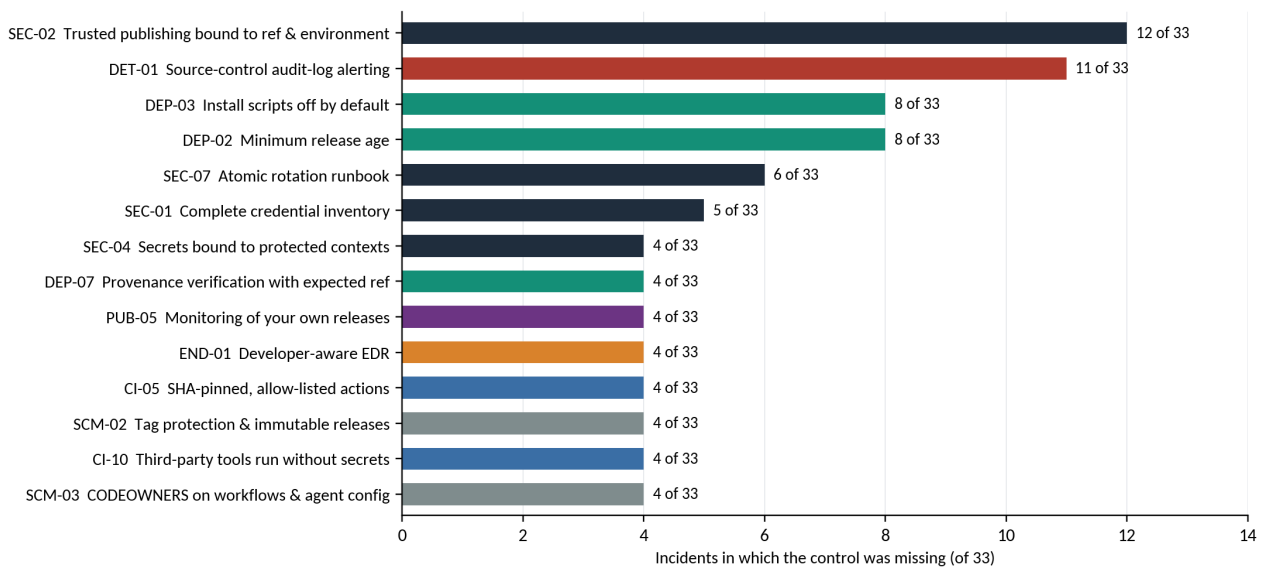


Figure 9.1. The 14 controls missed most often across 33 incidents. Colours show the control domain: navy SEC, red DET, green DEP, purple PUB, orange END, blue CI, grey SCM.

The ranking has a clear shape. Credential controls (SEC) dominate the top, because nearly every incident started with a token. Detection of source-control events (DET-01) is second, because tag rewrites, workflow additions, visibility changes and branch bursts were all visible in audit logs that few organisations watched. Dependency-intake controls (DEP-02, DEP-03, DEP-07) protect consumers who had no control over the upstream breach.

9.3 Incident-to-control matrix

Incident	Date	Missed controls
MemTensor	23 Sep 2026	SEC-04, SCM-04, CI-09, SEC-02, DEP-08, DET-01
CrowdSec downstream	disclosed 17 Sep 2026	TPR-02, DET-01, SEC-05, SCM-06
ChainDrop	4 Aug 2026	SCM-01, PUB-02, DEP-02, DEP-03, RSP-02, DET-09
Miasma wave 2	24 Jun 2026	CI-05, END-03, SCM-03
Mastra / easy-day-js	17 Jun 2026	ID-03, TPR-01, PUB-01, DEP-07, DEP-05, PUB-05
Hades	7 Jun 2026	SEC-02, DEP-08, RSP-06
Microsoft / Azure orgs	5 Jun 2026	SCM-01, ID-04, SCM-03, END-03, DET-01
Miasma (Red Hat)	1 Jun 2026	SEC-02, SCM-01, SCM-03, DEP-07, ID-04, SCM-06
DurableTask	19 May 2026	SEC-02, END-04, DET-06
AntV	18 May 2026	PUB-04, PUB-05, DEP-02, DEP-03
Actions-cool	18 May 2026	CI-05, SCM-02, DET-01
Nx Console	18 May 2026	PUB-01, PUB-06, SEC-06, END-02
UiPath / Mistral	11 May 2026	SEC-02, END-05, SEC-07
TanStack	11 May 2026	CI-06, CI-07, CI-03, CI-09, SEC-02, CI-08, CI-01
Cemu	7 May 2026	ART-03, SCM-02, ID-04, DET-01
SAP (Mini Shai-Hulud)	29 Apr 2026	CI-04, SEC-04, SEC-02, SEC-08
Bitwarden CLI	22 Apr 2026	SEC-01, DET-07, DEP-03, END-01
KICS Docker images	22 Apr 2026	SEC-01, ART-01, DET-07, DEP-02
fairwords / CanisterSprawl	8 and 22 Apr 2026	SEC-07, SEC-01, END-05, DEP-03
Telnyx	27 Mar 2026	SEC-02, CI-10, DEP-07, DEP-08
LiteLLM	24 Mar 2026	SEC-02, SCM-03, SEC-04, SCM-06, DET-01
Checkmarx KICS Action	23 Mar 2026	CI-05, SCM-02, ID-05, CI-10, DET-01
Docker Hub persistence	22 Mar 2026	ART-01, ART-02, SCM-04, SEC-01, DET-01
CanisterWorm	20 Mar 2026	SEC-02, CI-10, DEP-02, DEP-03, PUB-05
Spam flood	20 Mar 2026	RSP-03
Trivy main compromise	19 Mar 2026	CI-05, SCM-02, PUB-02, CI-10, CI-08, SEC-07, DET-01
Trivy precursor	27 Feb 2026	TPR-01, CI-01, ID-05, SEC-07, DET-01
Trust Wallet	24 Dec 2025	SEC-01, PUB-06, PUB-02, PUB-07, DET-10
Shai-Hulud 2.0	24 Nov 2025	SEC-07, CI-07, DET-01, DEP-03, END-01, END-06, DEP-02
GlassWorm (all waves)	from 17 Oct 2025	END-02, SCM-05, PUB-06, END-05, DET-09, END-01, SEC-06
Shai-Hulud	15 Sep 2025	SEC-02, SEC-06, PUB-04, DEP-03, DEP-02, END-01, DET-09, SEC-07
chalk / debug	8 Sep 2025	ID-01, PUB-01, DEP-02, DEP-07, PUB-05
Nx s1ngularity	26 Aug 2025	CI-01, CI-02, CI-03, SEC-02, SEC-04, DEP-03, DEP-02, END-03

9.4 Reading the matrix as a publisher and as a consumer

The same incident looks different depending on which side you are on. A **publisher** (any organisation that ships packages, actions, images, extensions or binaries) needs the CI, SCM, SEC and PUB controls, because those decide whether its release path can be hijacked. A **consumer** (every organisation that installs third-party code) needs the DEP, END and DET controls, because the upstream breach is outside its control and only intake, isolation and detection limit the damage.

Most organisations are both. The Trivy cascade turned consumers into publishers of malware within days: organisations that only ran a scanner lost the tokens that published the next wave. Part IV therefore treats publisher and consumer controls as one programme.

PART IV

Building the programme from scratch

CHAPTER 10

Principles, ownership and the maturity model

A supply chain programme built from the record of this year rests on eight principles, is owned by five teams, and matures through four levels. Chapters 11 to 13 define 73 controls across eleven domains; this chapter explains how they fit together and in what order to build them.

10.1 Eight principles

- 1. Credentials first.** Nearly every incident began with a token. Eliminate long-lived publishing and cloud credentials before investing in anything else.
- 2. Every automatic execution point is intake.** Install hooks, import-time code, .pth files, native builds, workflow triggers, editor tasks and agent hooks all run third-party code. Govern them as one surface.
- 3. Bind identity to intent.** A trusted-publishing token or a provenance statement must prove that the intended workflow ran on the protected ref, in the protected environment, with the required approval, not merely that some workflow ran.
- 4. Consuming is exposure.** If a compromised tool ran in your pipeline, your pipeline's secrets are stolen. Plan containment for consumers, not only for the breached vendor.
- 5. Reference by content, not by name.** Tags and version names can move. Commit SHAs and image digests cannot.
- 6. Time is a control.** Short credential lifetimes shrink what an attacker can steal; minimum release ages give the community time to catch what an attacker publishes.
- 7. Watch the machine paths.** Attackers used tokens, workflows, tags, caches and release APIs, all of which leave audit-log events. Alert on those events, not only on human logins.
- 8. Respond to an adversary who is watching.** Expect spam floods, revocation-triggered wipers and prompt injection. Snapshot before you revoke, coordinate out of band, and keep AI analysis on a short leash.

10.2 Who owns what

Team	Owns	Control domains
CISO and security leadership	Programme, risk acceptance, metrics, response authority	All; accountable for RSP
Platform / DevOps engineering	CI/CD, runners, build environments, release pipelines, artefact registries	CI, PUB, ART
Application security	Source-control policy, workflow review, dependency intake, provenance	SCM, DEP, PUB (shared)
Security operations	Detection, threat intelligence, incident response	DET, RSP, SEC (rotation)
IT and identity	Developer endpoints, identity provider, offboarding, SaaS and OAuth governance	ID, END, TPR

Legal, procurement and communications have defined roles in response (see the Supply Chain IR RACI in the companion toolkit) and in third-party terms (TPR-03).

10.3 The four maturity levels

Domain	Level 1: Foundational	Level 2: Managed	Level 3: Hardened	Level 4: Resilient
Identity and access (ID)	MFA everywhere; named owners for bot accounts	Phishing-resistant MFA; quarterly access reviews	Inactive access auto-removed; time-bound contributor write	Separate publishing identities; continuous access analytics
Source control (SCM)	Default-branch protection	Rulesets incl. tags; CODEOWNERS on workflows	All release refs protected; signed commits; visibility locked	Immutable releases everywhere; automated policy drift checks
CI/CD (CI)	Workflow linting; read-only default token	SHA pinning enforced; no secrets to forks	Protected publish environments; ephemeral runners; cache isolation	Egress allow-lists on all privileged jobs; runtime monitoring
Secrets and credentials (SEC)	Inventory started; secret scanning	Trusted publishing for top packages; OIDC to cloud	No long-lived publish or cloud tokens anywhere	Automated rotation; credential use tied to workload identity
Dependency intake (DEP)	Lockfiles; frozen installs	Minimum release age; install scripts off	Internal proxy with quarantine; provenance verification	Continuous SBOM-to-intel matching; new-dependency review
Publishing (PUB)	Publish from CI only	Provenance on every release	Two-person release approval; package-scoped rights	Reproducible builds verified against published artefacts
Artefacts (ART)	Images from known registries	Pull by digest	Signature verification at admission	Signed release assets verified by all consumers
Endpoints and tools (END)	Managed devices with EDR	Extension allow-list	AI-agent and workspace-trust policy; sandboxed installs	Separate publishing workstations; behaviour analytics
Third parties (TPR)	OAuth app inventory	Token-based offboarding	Contributor and vendor access time-bound	Supplier scoring drives intake policy
Detection (DET)	Audit logs collected	Alerts on high-risk SCM events	Honeytokens; CI runtime telemetry	Retired-credential watch; correlated detections across SCM, CI, cloud and endpoint
Response (RSP)	Playbook exists	Blast-radius and rotation runbooks rehearsed	Out-of-band channel; snapshot-before-revoke	Twice-yearly exercises with measured improvement

Most organisations starting from scratch should aim to reach Level 2 across all domains within 90 days and Level 3 within a year. Chapter 14 sequences that work.

10.4 How to read the control chapters

Each control has an ID, a type (Preventive, Detective or Responsive), a priority (P1 to P3), and a short implementation note covering the major platforms. P1 controls address the most frequent or most damaging failures in Chapter 9 and should be in place before the programme claims Level 2. Where a control's implementation differs by platform, GitHub is described first because it hosted most of the incidents, followed by GitLab, CircleCI and the major registries.

CHAPTER 11

Preventive controls

Preventive controls stop the attacker's first step or make it useless. This chapter defines 57 preventive controls in nine domains. Each table gives the control, its priority, how to implement it on the major platforms, and the incidents in Part II it would have stopped or contained.

11.1 Identity and access (ID)

Attackers repeatedly used real accounts: phished maintainers, compromised contributors and dormant members. Identity controls decide whether a stolen password, session or membership is enough to publish.

ID	Control	Priority	How to implement	Would have stopped
ID-01	Phishing-resistant MFA on every account that can push, publish, deploy or administer	P1	Enforce passkeys or hardware security keys at the organisation level for the source host, every package registry, cloud consoles, CI platforms and app and extension stores. Disable SMS and, for privileged accounts, one-time-password fallback.	chalk/debug; reduces account-takeover paths in Cemu, Microsoft orgs, ChainDrop
ID-02	Separate identities for privileged publishing	P2	Publishing and organisation-admin rights do not sit on the account a developer uses daily to browse, install packages and run agents. Prefer no human publishing at all (PUB-01).	Nx Console, Microsoft orgs, Miasma
ID-03	Access reviews with automatic expiry	P1	Review organisation, team, repository, registry-scope and marketplace-namespace membership quarterly. Remove members inactive for 90 days. Alert scope owners when a member's email address or MFA settings change.	Mastra; Trivy precursor
ID-04	Least-privilege, time-bound write access	P2	Grant contributors write access per repository, for a defined period. Avoid organisation-wide write roles for individuals. Require approval to extend.	Microsoft orgs, Miasma, Cemu
ID-05	Governed service and bot identities	P1	Inventory every bot and service account with a named owner. Prefer GitHub Apps with short-lived installation tokens over bot PATs. Scope each to the repositories and permissions it needs; never let humans sign in as a bot.	Trivy precursor, Trivy main, Checkmarx KICS

11.2 Source control governance (SCM)

Source-control settings decide who can change code, workflows, tags and releases, and whether anyone notices. Most were left at defaults that protect only the default branch.

ID	Control	Priority	How to implement	Would have stopped
SCM-01	Rulesets on default and release branches with no silent bypass	P1	Require pull requests, at least one approving review from someone other than the author, signed commits and passing checks. Block force-pushes and deletion. Do not exempt maintainers; if bypass is needed, make it an audited, alerted exception.	ChainDrop, Miasma, Microsoft orgs
SCM-02	Tag protection and immutable releases	P1	Tag rulesets that block creation, update and deletion of release tags except by the release workflow's identity. Enable immutable releases where the platform supports them, so assets and tags cannot change after publication.	Trivy, Checkmarx KICS, Actions-cool, Cemu

ID	Control	Priority	How to implement	Would have stopped
SCM-03	CODEOWNERS on everything that executes	P1	Require security or platform review for changes to .github/workflows, CI configuration, build scripts, package manifests and lockfiles, and AI-agent and editor configuration directories such as .claude, .cursor and .vscode.	LiteLLM, Miasma, Microsoft orgs, Miasma wave 2
SCM-04	Lock down visibility, creation and release-pattern branches	P2	Organisation policy so members cannot make repositories public, create public repositories or fork private ones. Restrict who can create branches matching release patterns (release/sc/, v*). Block deletion of branches that ran workflows with secrets.	Docker Hub persistence (repo exposure), MemTensor
SCM-05	Hidden-character detection	P2	A required check that fails on invisible, private-use and bidirectional Unicode in source, workflow and package files, with an explicit allow-list for legitimate uses.	GlassWorm
SCM-06	Token and OAuth-app policy for the source host	P1	Require approval for fine-grained personal access tokens, set a maximum lifetime (for example 30 days), block classic tokens, and require admin approval for OAuth apps. Review grants quarterly.	LiteLLM, Miasma, CrowdSec downstream

11.3 CI/CD pipeline hardening (CI)

CI runners hold the most powerful secrets an organisation has and run on events outsiders can trigger. This domain carries the most technical detail because it was the most exploited.

ID	Control	Priority	How to implement	Would have stopped
CI-01	No privileged workflows that run untrusted code	P1	Ban pull_request_target and workflow_run workflows that check out, build or execute pull-request code. Where such a trigger is needed, run only trusted code from the base branch and pass PR data as inert input. Enforce with a workflow linter (for example zizmor or actionlint) as a required check.	Nx, Trivy precursor, TanStack
CI-02	No interpolation of untrusted context into scripts	P1	Never place event fields such as titles, branch names or commit messages directly in run commands. Pass them through environment variables and quote them. The linter in CI-01 detects this.	Nx
CI-03	Least-privilege job tokens	P1	Set the organisation default GITHUB_TOKEN to read-only. Declare permissions per job. Grant id-token: write only to the single publish or deploy job. On GitLab, restrict CI_JOB_TOKEN access with an allow-list.	Nx, TanStack
CI-04	No secrets for forked contributions	P1	Require approval before workflows run for outside contributors. Never expose secrets to fork builds on any platform: on CircleCI turn off “Pass secrets to builds from forked pull requests”; on GitLab keep fork pipelines in the fork project and use protected variables.	SAP, Trivy precursor
CI-05	Immutable, allow-listed third-party actions	P1	Pin every third-party action, reusable workflow, CircleCI orb and job container to a full commit SHA or image digest. Enforce SHA pinning and an allow-list of approved actions in organisation policy. Use a bot such as Renovate or Dependabot to propose SHA updates with a cooldown.	Trivy, Checkmarx KICS, Actions-cool, Miasma wave 2
CI-06	Cache isolation	P1	Release and publish workflows do not restore caches written by pull-request or fork-triggered runs. Use separate cache keys or no cache at all for release jobs, and never let pull_request_target write to a cache other workflows read.	TanStack
CI-07	Ephemeral runners and runner registration control	P1	Run privileged jobs on fresh, single-use runners. Never attach self-hosted runners to public repositories. Restrict runner registration to administrators through runner groups and alert on every new runner.	TanStack, Shai-Hulud 2.0

ID	Control	Priority	How to implement	Would have stopped
CI-08	Egress control on runners	P2	Allow-list outbound destinations for publish, release and deploy jobs at minimum, using a runner egress agent or network policy. Block blockchain RPC endpoints and paste sites unless explicitly needed.	Trivy, TanStack, CanisterWorm, ChainDrop
CI-09	Isolated, approved publish step	P1	Build and test in one job with no secrets. Publish in a separate job that runs alone, on a fresh runner, in a protected environment with required reviewers who are not the author, and with deployment rules limited to the default branch or protected tags. Do not let earlier steps write to GITHUB_ENV, BASH_ENV or PATH in that job.	TanStack, MemTensor, SAP
CI-10	Tools run without secrets	P1	Run scanners, linters and other third-party tools in jobs that have read-only tokens and no publish, deploy or cloud credentials. A compromised tool then has nothing worth stealing.	Trivy consumers, CanisterWorm, Telnyx, Checkmarx KICS

11.4 Secrets and credentials (SEC)

The single largest category of missed controls. The goal is an organisation with no long-lived credential that can publish, push or deploy, and a complete list of the ones that remain.

ID	Control	Priority	How to implement	Would have stopped
SEC-01	A complete credential inventory	P1	Record every token, key and certificate that can publish, push, deploy or administer: owner, scope, systems that hold it, expiry and rotation steps. Cover package registries, container registries, source hosts, cloud, CI platforms, app and browser-extension stores, editor marketplaces, chat and monitoring webhooks. Rotation (SEC-07) works from this list.	Trust Wallet, KICS Docker, Bitwarden CLI, fairwords
SEC-02	Trusted publishing bound to ref and environment	P1	Replace stored registry tokens with trusted publishing on npm, PyPI, RubyGems, crates.io and others that support it. Bind the trust to one repository, one workflow file and one protected environment whose deployment rules allow only the default branch or protected tags. Then disallow token publishing on the package.	12 of 33 incidents, including Nx, Shai-Hulud, CanisterWorm, LiteLLM, Telnyx, Miasma, Hades
SEC-03	Short-lived cloud credentials through OIDC	P1	Federate CI to cloud providers with OIDC and narrow subject claims (repository, environment, ref). Remove static cloud keys from CI secrets and developer machines.	Reduces damage in Trivy, AntV, DurableTask, ChainDrop
SEC-04	Secrets bound to protected contexts	P1	Store publish and deploy secrets in protected environments (GitHub), protected variables (GitLab) or restricted contexts (CircleCI) that only protected refs can reach. A new branch or a new workflow file must not be able to read them.	Nx, LiteLLM, SAP, MemTensor
SEC-05	Secret scanning with push protection	P2	Enable secret scanning and push protection on every repository, including private ones, and scan history. Scan built packages before publishing.	CrowdSec downstream (secrets in cloned repos)
SEC-06	No long-lived tokens on developer machines	P1	Remove registry tokens from .npmrc and .pypirc and long-lived source-host tokens from shells. Use SSO-backed credential helpers with short sessions. Combined with PUB-01, developers should not need publish tokens at all.	Shai-Hulud, Nx Console, GlassWorm
SEC-07	Atomic rotation runbook	P1	A pre-written, rehearsed procedure that rotates every credential in a blast radius in one coordinated window and verifies each one, as described in the companion Atomic Rotation Protocol.	Trivy, Shai-Hulud 2.0, fairwords, UiPath/Mistral
SEC-08	Signing keys in hardware or keyless signing	P2	Keep code-signing and app-signing keys in an HSM, cloud KMS or a signing service, or use keyless signing. Never store a signing certificate as a plain CI secret.	SAP (iOS signing certificate)

11.5 Dependency intake (DEP)

Intake controls protect consumers who cannot prevent an upstream breach. Their job is to keep a malicious version out of builds and endpoints for long enough that it is found and pulled.

ID	Control	Priority	How to implement	Would have stopped
DEP-01	Lockfiles and frozen installs	P1	Commit lockfiles for every application and CI tool. Install with <code>npm ci</code> , <code>pnpm install --frozen-lockfile</code> , <code>yarn install --immutable</code> , or <code>pip install --require-hashes / uv sync --locked</code> . Never run open-ended installs in CI.	Most registry worms, for builds that did not update
DEP-02	Minimum release age	P1	Refuse dependency versions younger than a set age, for example seven days, with a documented fast path for security fixes. Use the release-age settings in <code>pnpm</code> , <code>Yarn</code> and <code>Bun</code> , <code>Renovate's</code> <code>minimumReleaseAge</code> , <code>Dependabot's</code> <code>cooldown</code> , or <code>uv's</code> <code>exclude-newer</code> .	<code>chalk/debug</code> , <code>Shai-Hulud</code> , <code>Shai-Hulud 2.0</code> , <code>CanisterWorm</code> , <code>AntV</code> , <code>ChainDrop</code> , <code>KICS Docker</code>
DEP-03	Install scripts off by default	P1	Disable dependency lifecycle scripts (<code>npm</code> <code>ignore-scripts</code> , <code>pnpm's</code> default with an <code>onlyBuiltDependencies</code> allow-list, <code>Bun</code> <code>trustedDependencies</code> , <code>Yarn</code> <code>enableScripts: false</code>). Allow-list the few packages that genuinely need a build step.	<code>Nx</code> , <code>Shai-Hulud</code> , <code>Shai-Hulud 2.0</code> , <code>CanisterSprawl</code> , <code>Bitwarden CLI</code> , <code>AntV</code> , <code>ChainDrop</code>
DEP-04	Internal proxy registry with quarantine	P2	Route all installs through an internal mirror that screens new versions for malware, quarantines versions until they pass the release-age rule, and can block a version for the whole organisation in one action.	<code>TanStack</code> , <code>ChainDrop</code> and every fast-moving worm
DEP-05	New-dependency and maintainer-change review	P2	Flag updates that add a new direct or transitive dependency, a package under 30 days old, a change of maintainer, or a release that lacks provenance when its predecessors had it. Require human review for those.	<code>Mastra</code>
DEP-06	Build-time SBOM and live inventory	P1	Generate an SBOM for every build and keep a searchable inventory of what runs where, including CI tools, actions and developer tooling. Target: answer “are we affected?” for any package in under 15 minutes.	Speeds response in every incident
DEP-07	Provenance verification with expected ref	P2	Verify registry signatures and attestations at intake (<code>npm</code> audit signatures, <code>PyPI</code> attestations, <code>Sigstore</code> verification for images). Check that provenance names the expected repository and a release tag or default branch, not merely that it exists.	<code>Mastra</code> , <code>Miasma</code> , <code>chalk/debug</code> , <code>Telnyx</code>
DEP-08	Treat every execution point as code	P2	Scanners and reviewers treat <code>.pth</code> files, <code>binding.gyp</code> , <code>build.rs</code> , <code>import-time</code> module code and bundled binaries as executable. Prefer wheels over source builds (<code>pip --only-binary</code>) while still scanning wheels for <code>.pth</code> files.	<code>LiteLLM</code> , <code>Telnyx</code> , <code>Hades</code> , <code>Miasma</code> , <code>MemTensor</code>

11.6 Publishing and release integrity (PUB)

Publishing controls protect everyone downstream of you. They decide whether a stolen token, account or workflow can produce a release in your name.

ID	Control	Priority	How to implement	Would have stopped
PUB-01	Publish only from the release pipeline	P1	No human publishes from a laptop. Configure packages to require trusted publishing and disallow token-based publishing where the registry supports it.	<code>chalk/debug</code> , <code>Nx Console</code> , <code>Mastra</code>
PUB-02	Two-person release approval	P1	The publish job runs in a protected environment requiring approval from a maintainer other than the one who triggered it, with self-review prevented.	<code>Trivy</code> , <code>Trust Wallet</code> , <code>ChainDrop</code>
PUB-03	Provenance, signed tags and immutable releases	P2	Publish provenance attestations for every release, sign release tags, and make releases immutable once published. Document the expected provenance so consumers can verify it.	Gives consumers DEP-07

ID	Control	Priority	How to implement	Would have stopped
PUB-04	Package-scoped publishing rights	P2	No single identity can publish every package in a scope. Split rights per package or group, so one stolen credential cannot republish hundreds of packages.	Shai-Hulud, AntV
PUB-05	Monitor your own releases	P1	Subscribe to registry events for every package you publish. Alert when a version appears that did not come from the expected workflow, lacks provenance, adds a dependency or a binary, or arrives outside a release window.	chalk/debug, CanisterWorm, AntV, Mastra
PUB-06	Same controls for stores and marketplaces	P1	Apply PUB-01 to PUB-05 to browser-extension stores, editor marketplaces (VS Code Marketplace, OpenVSX), mobile app stores and container registries, which are often run by different teams.	Trust Wallet, GlassWorm, Nx Console, Trivy precursor
PUB-07	Reproducible builds and published-artefact verification	P3	Make builds reproducible and automatically compare each published artefact with a rebuild of its tagged source.	Trust Wallet, Cemu

11.7 Containers and release artefacts (ART)

ID	Control	Priority	How to implement	Would have stopped
ART-01	Pull by digest and verify signatures	P1	Reference images by digest in CI and deployment manifests. Enforce signature verification (Sigstore cosign or Notation) through admission policy in clusters and a verify step in CI.	Trivy images, Docker Hub persistence, KICS Docker
ART-02	Registry push only from the release pipeline	P1	Use organisation-scoped, least-privilege registry tokens held by the release pipeline. No personal access tokens can push to official namespaces.	Docker Hub persistence, KICS Docker
ART-03	Signed, verifiable release assets	P2	Publish signed checksums and attestations for every binary. Consumers verify before running. Combined with SCM-02, assets cannot be swapped silently.	Cemu

11.8 Developer endpoints, IDE extensions and AI tools (END)

The developer workstation is part of the build system. It holds tokens, runs installs and opens untrusted repositories.

ID	Control	Priority	How to implement	Would have stopped
END-01	Managed endpoints with developer-aware EDR	P1	Every machine that can push or publish is managed and runs EDR with detections for package managers spawning shells, downloading runtimes, running secret scanners or reading other processes' memory (see DET-04).	Shai-Hulud, Shai-Hulud 2.0, GlassWorm, Bitwarden CLI
END-02	Extension allow-list and staged updates	P1	Enforce an allow-list of editor extensions through enterprise policy (for example VS Code's extensions.allowed setting). Mirror approved extensions privately and delay updates.	GlassWorm, Nx Console, Trivy extension
END-03	AI-agent and workspace-trust policy	P1	Approve which AI coding agents may be used. Prohibit permissive flags that skip approval. Require explicit trust before an editor or agent runs repository-supplied tasks, hooks or settings; keep Workspace Trust on and automatic tasks off.	Nx, Microsoft orgs, Miasma wave 2
END-04	Sandboxed installs for untrusted code	P2	Install and test unfamiliar packages in development containers or disposable VMs without access to the host's credentials. Keep publishing on a separate, minimal machine or profile.	DurableTask, all install-time stealers
END-05	No vaults, wallets or browser passwords within reach	P2	Lock password-manager CLI sessions, disallow browser-stored passwords on developer machines, and keep cryptocurrency wallets off any machine with production publishing rights.	CanisterSprawl, UiPath/Mistral, GlassWorm
END-06	Backups of developer home directories	P3	Back up source and configuration in home directories so a wiper costs hours, not weeks.	Shai-Hulud 2.0, ChainDrop

11.9 Third parties, OAuth and offboarding (TPR)

ID	Control	Priority	How to implement	Would have stopped
TPR-01	Token-driven offboarding	P1	When a person leaves or a contributor goes inactive, revoke every token and membership they hold on source hosts, registries, marketplaces, CI and cloud, using the SEC-01 inventory rather than only the identity provider.	Trivy precursor, Mastra
TPR-02	OAuth and integration governance	P2	Inventory OAuth apps, GitHub Apps and webhooks; approve new ones; review and rotate quarterly; alert on bulk clone or bulk API activity by an integration.	CrowdSec downstream
TPR-03	Contractual supply chain obligations	P3	Require critical suppliers to notify you of supply chain compromises within a set time and to share affected versions, windows and indicators.	Shortens every downstream response
TPR-04	Supplier and project health scoring	P3	Score critical open-source dependencies and tools on maintainer count, trusted-publishing use, provenance, branch protection and security policy (for example with OpenSSF Scorecard), and use the score in intake decisions.	Prioritises DEP controls

CHAPTER 12

Detective controls

Detection was the second most frequently missed control family. In almost every incident the attacker's first step produced a log event (a tag rewrite, a new workflow file, a publish without provenance, a runner reading another process's memory) that nobody was alerting on. This chapter defines ten detective controls and 24 concrete detection use cases.

12.1 Detective controls

ID	Control	Priority	How to implement	Would have caught
DET-01	Source-control audit-log alerting	P1	Stream organisation and enterprise audit logs from the source host to the SIEM. Alert on the high-risk events in use cases 1 to 9 below. Retain at least one year.	11 of 33 incidents, including Trivy, Docker Hub persistence, Cemu, Microsoft orgs, MemTensor
DET-02	Registry publish monitoring	P1	Monitor registry events for every package you publish (PUB-05), and subscribe to malicious-package feeds for packages you consume.	chalk/debug, CanisterWorm, AntV, Mastra
DET-03	CI runtime telemetry	P2	Collect process, network and file events from runners for privileged jobs, with a runner security agent or an EDR sensor on self-hosted runners.	Trivy, TanStack, Actions-cool, MemTensor
DET-04	Developer-endpoint detections	P1	Behavioural detections on developer machines for the install-time and editor-time behaviours in use cases 15 to 20.	Shai-Hulud, GlassWorm, Bitwarden CLI, Nx Console
DET-05	Honeytokens	P2	Plant canary cloud keys, registry tokens and source-host tokens in CI secrets, environment files, .npmrc files and internal repositories. Any use is a confirmed compromise.	Every stealer in this book, at first use
DET-06	Cloud control-plane detections for build and developer identities	P2	Alert when a CI or developer identity calls services it never uses (Systems Manager, Secrets Manager bulk reads, Kubernetes exec) or authenticates from a new network.	DurableTask, AntV, ChainDrop
DET-07	Retired-credential watch	P1	For 90 days after any rotation, alert on every attempt to use a revoked or replaced credential, including against registries and container hubs.	KICS Docker, Bitwarden CLI, fairwords
DET-08	Intelligence matched to inventory	P1	Ingest OSV, GitHub advisories, CISA KEV and vendor malicious-package feeds and match them automatically against the DEP-06 inventory, opening a ticket with affected services.	Speeds every response
DET-09	Exfiltration-channel detection	P2	Detect public repositories created by employee accounts with known campaign names; commits of JSON blobs to new repositories; and traffic to blockchain RPC endpoints, paste sites and newly registered domains from CI and developer machines.	Shai-Hulud, GlassWorm, ChainDrop
DET-10	Brand and look-alike domain monitoring	P3	Monitor new domain registrations resembling your brand and products.	Trust Wallet

12.2 Detection use cases

The use cases below are written so a detection engineer can implement them. Each names the data source, the logic in plain language and the incidents in which it would have fired. Thresholds are starting points to tune.

#	Use case	Data source	Logic	Would have fired in
1	Tag rewrite burst	Source-host audit log	More than three release tags updated or force-pushed in one repository within 10 minutes, or any tag update by an identity other than the release workflow	Trivy, Checkmarx KICS, Actions-cool
2	Release asset replaced without a workflow	Audit log + CI logs	A release asset uploaded or replaced with no corresponding workflow run in the same window	Cemu
3	New workflow file outside review	Audit log + git events	A file under .github/workflows created or changed on a non-default branch and executed before any pull request	LiteLLM, Miasma, MemTensor
4	Short-lived branch that ran secrets	Audit log	A branch created, used by a workflow that accessed secrets or an environment, and deleted within 30 minutes	MemTensor
5	Mass push across branches or repositories	Audit log	One identity pushes to more than 20 branches or 10 repositories within an hour	Microsoft orgs
6	Repository exposure	Audit log	Repository visibility changed to public, or more than five repositories renamed within 10 minutes	Docker Hub persistence (Aqua org defacement)
7	New self-hosted runner	Audit log	Any runner registration outside an approved change window, or with an unusual name	Shai-Hulud 2.0
8	Risky token and OAuth activity	Audit log	New fine-grained or classic token created for an organisation, OAuth app authorised, or bulk clone by an integration or user	LiteLLM, Miasma, CrowdSec downstream
9	Unsigned direct push followed by release	Audit log + registry	An unsigned commit pushed directly to the default branch followed by a release or publish within 30 minutes	ChainDrop
10	Publish from an unexpected path	Registry events / provenance	A version of your package published without provenance, from a token, from a non-default ref, or outside the release workflow	chalk/debug, Mastra, Telnix, Miasma
11	Publish-rate anomaly	Registry events	More than five versions of your packages published within 10 minutes, or by a dormant account	TanStack, AntV, Mastra
12	Dependency appears in feed	Intel feed + SBOM	A version in your inventory appears in a malicious-package feed or advisory	All registry incidents
13	Runner process memory access	CI runtime telemetry	Any process on a runner reading /proc/\mem or maps of the Runner.Worker or another build process	Trivy, TanStack, Actions-cool
14	Runner egress to new destinations	CI runtime / network	A privileged job connects to a domain first seen that day, a blockchain RPC endpoint or a paste site	Trivy, CanisterWorm, ChainDrop
15	Package install spawns a secret scanner	Endpoint EDR	TruffleHog, gitleaks or similar launched with npm, node, bun or python as an ancestor	Shai-Hulud
16	Install-time runtime download	Endpoint EDR / CI telemetry	A package manager process downloads and executes a runtime binary such as Bun, Deno or a Go executable	Shai-Hulud 2.0, Bitwarden CLI, AntV, MemTensor
17	Install or import process reads credential files	Endpoint EDR	A package-manager or Python process reads .npmrc, .pypirc, cloud credential files, SSH keys or browser credential stores	Every stealer in this book

#	Use case	Data source	Logic	Would have fired in
18	Install-time AI agent invocation	Endpoint EDR	A package install launches an AI coding CLI, especially with permission-skipping flags	Nx
19	Editor or extension host unusual network	Endpoint EDR / network	The editor's extension host connects to blockchain RPC, calendar APIs or opens a listening SOCKS or VNC port	GlassWorm
20	Home-directory wipe	Endpoint EDR	Recursive deletion of a user's home directory by a process descended from a package manager or Node.js	Shai-Hulud 2.0, ChainDrop
21	Honeytoken use	Cloud / registry / source-host logs	Any authentication with a canary credential	All stealers
22	Retired credential used	Registry, cloud and source-host logs	Any attempt with a credential rotated in the last 90 days	KICS Docker, Bitwarden CLI, fairwords
23	Build identity calls unusual cloud APIs	Cloud audit logs	A CI or developer role calls Systems Manager, Secrets Manager list/get in bulk, or Kubernetes exec, for the first time	DurableTask, AntV
24	Campaign exfiltration repositories	Source-host search / partner alerts	A repository named like known campaigns ("Shai-Hulud", "s1ngularity") or described with campaign strings appears under an employee or bot account	Nx, Shai-Hulud, Shai-Hulud 2.0, Bitwarden CLI

12.3 Telemetry you need before you can detect anything

Source	What to collect	Owner
Source host	Organisation and enterprise audit logs, git events where available, secret-scanning alerts	AppSec / SecOps
CI platform	Workflow run metadata, job logs, environment approvals, runner registration	Platform engineering
CI runners	Process, network and file events for privileged jobs	Platform engineering / SecOps
Package and container registries	Publish events and webhooks for your packages; token activity where exposed	AppSec
Cloud	Control-plane audit logs for every role used by CI and developers	Cloud security
Developer endpoints	EDR process, file and network telemetry	IT / SecOps
Intelligence	Malicious-package and advisory feeds	SecOps

12.4 Making detection actionable

Each use case needs a named responder and a first action. For use cases 1 to 11 the first action is to freeze the affected release path; for 12 to 20 it is to isolate the machine or runner and snapshot it before any token is revoked (RSP-02); for 21 to 24 it is to start the 30-Minute Blast Radius Assessment. A detection that fires into a queue nobody reads for a day has, on this year's evidence, missed most of the attacker's window.

CHAPTER 13

Response readiness

Response controls decide how much a breach costs once prevention and detection have done their part. The year's evidence sets the standard: a credential stolen today is likely to be used within eight days and may be used five weeks later, and the attacker may be watching the response. Six responsive controls and a four-stage response model follow.

13.1 Responsive controls

ID	Control	Priority	How to implement	Lesson from
RSP-01	A supply-chain-specific incident playbook	P1	Adopt a playbook covering the first 30 minutes (blast radius), credential rotation, ownership and communications. The companion toolkit provides a 30-Minute Blast Radius Assessment, an Atomic Rotation Protocol, a Supply Chain IR RACI and a CISO Communication Playbook card.	Every incident
RSP-02	Snapshot and isolate before revoking	P1	When a machine or runner is infected, isolate it from the network and capture memory and disk before revoking the tokens it held, because some payloads wipe the host on revocation. Revoke from a clean machine.	Shai-Hulud 2.0, ChainDrop
RSP-03	Out-of-band coordination and private advisories	P2	Keep a pre-agreed channel for maintainers, major consumers and vendors that does not depend on the compromised platform. Publish through a security-advisory mechanism, and be ready to lock public threads.	Spam floods during Trivy and LiteLLM disclosure
RSP-04	"Did we ship it?" check	P1	Within the first hour, determine whether anything you publish was built, signed or released during the exposure window, and if so treat the incident as customer-facing.	Trivy consumers that became publishers of the next wave; LiteLLM; Telnyx
RSP-05	Exercises against real incidents	P2	Run tabletop exercises at least twice a year using scenarios from Part II, timed and scored against the 30-minute and 24-hour targets in 13.2.	Decision failures throughout the year
RSP-06	Guardrails for AI-assisted analysis	P3	When AI tools help analyse samples or logs, treat sample content as untrusted input, keep a human decision on every conclusion, and never let analysis tools act on systems.	Hades

13.2 The response model

- 1. First 30 minutes: scope.** Name an incident commander and move to an out-of-band channel. Establish which environments ran the affected version, which credentials those environments could reach, whether anything you ship was built in the window, and which regulatory clocks may have started.
- 2. First 24 hours: contain.** Snapshot affected hosts and runners. Freeze affected release paths. Rotate every reachable credential in one coordinated, verified operation. Notify regulators where a clock requires it, and coordinate privately with affected vendors and customers before any public statement.
- 3. First week: eradicate and recover.** Rebuild from clean runners and pinned, verified inputs. Remove malicious versions from internal mirrors and caches. Hunt for persistence such as new runners, workflow files, agent configuration files and new OAuth grants. Publish a factual update.

- 4. Days 8 to 90: watch and harden.** Alert on any use of retired credentials (DET-07). Track the hardening items the incident exposed to closure with owners and dates. Review the incident against this book's control matrix.

13.3 Regulatory clocks to check in the first 30 minutes

Obligations depend on jurisdiction, sector and the data involved, and must be confirmed with counsel. The table lists the clocks most often relevant to a supply chain compromise.

Framework	Applies to	Clock
GDPR Art. 33	Personal data of people in the EU	72 hours from awareness to the supervisory authority
NIS2	Essential and important entities in the EU	24-hour early warning, 72-hour notification
DORA	EU financial entities	Initial notification within hours of classifying a major incident
US SEC Form 8-K, Item 1.05	US-listed companies	4 business days from determining an incident is material
CIRCA	US critical infrastructure (when its rules apply)	72 hours for covered incidents
PCI DSS	Cardholder data environments	Immediately, per card-brand and acquirer requirements
Contracts and insurance	Customers, partners, cyber insurers	As written in each agreement or policy

13.4 Publisher and consumer responses differ

A **consumer** that ran a compromised dependency must assume every secret reachable from the affected environments is stolen, even if no exfiltration is proven, and rotate them all. A **publisher** whose release path was hijacked must also identify every version published in the window, remove or deprecate it, notify downstream users through advisories and registry mechanisms, and explain how consumers can verify the clean versions. Many organisations will be both in the same incident, as the Trivy cascade showed.

CHAPTER 14

Implementation roadmap and metrics

An organisation starting from nothing can close the most-exploited gaps in 30 days, reach Maturity Level 2 in 90 days and Level 3 in a year. The roadmap below sequences all 73 controls into five phases, ordered by how often each gap was exploited and how quickly it can be closed.

14.1 Five phases

Phase	Window	Goal	Controls
0. Mobilise	Weeks 0–2	Owners named, inventories started, cheapest high-value settings changed	ID-01, ID-05, CI-03, CI-04, SEC-01 (start), DEP-01, DEP-03, DEP-06 (start), DET-01 (log collection), SEC-05 (enable), RSP-01
1. Close the doors	Days 15–30	Remove the entry points used most often	SEC-02 (top publish paths), SEC-06, CI-01, CI-02, CI-05, SCM-02, DEP-02, ID-03, TPR-01, PUB-05, DET-01 (alerts), DET-02, DET-07, DET-08
2. Harden the build	Days 31–90	Make release paths approval-gated and isolated	SEC-02 (all paths), SEC-03, SEC-04, SEC-07 (rehearsed), CI-06, CI-07, CI-09, CI-10, SCM-01, SCM-03, SCM-06, PUB-01, PUB-02, PUB-06, ART-01, ART-02, END-01, END-02, END-03, DET-04, DET-05, RSP-02, RSP-04
3. Verify and isolate	Days 91–180	Verify what you consume and contain what runs	DEP-04, DEP-05, DEP-07, DEP-08, CI-08, PUB-03, PUB-04, ART-03, END-04, END-05, SCM-04, SCM-05, ID-02, ID-04, TPR-02, DET-03, DET-06, DET-09, RSP-03, RSP-05
4. Build resilience	Days 181–365	Reach Level 3 and start Level 4	SEC-05 (full history), SEC-08, PUB-07, END-06, TPR-03, TPR-04, DET-10, RSP-06, automated rotation

Phase 0 and Phase 1 together include at least one of the missed controls in 32 of the 33 incidents in Chapter 9. If a programme can only fund one quarter of work, it should fund those two phases.

14.2 The first 30 days in detail

- ☐ Enforce phishing-resistant MFA for every account with push, publish, deploy or admin rights (ID-01).
- ☐ Set the organisation default workflow token to read-only and add per-job permissions to release workflows (CI-03).
- ☐ Turn off secrets for forked pull requests on every CI platform and require approval for outside contributors (CI-04).
- ☐ Make frozen installs and lockfiles mandatory in CI; disable dependency install scripts by default (DEP-01, DEP-03).
- ☐ Start the credential inventory with every registry, container hub, store and marketplace token (SEC-01).
- ☐ Stream source-host audit logs to the SIEM and turn on use cases 1, 3, 6, 7 and 9 (DET-01).
- ☐ Move the ten most-downloaded packages you publish to trusted publishing bound to a protected environment, and disallow token publishing on them (SEC-02).
- ☐ Add a workflow linter as a required check and fix every `pull_request_target` and script-injection finding (CI-01, CI-02).
- ☐ Pin every third-party action to a commit SHA and turn on SHA-pinning enforcement (CI-05).
- ☐ Protect release tags with rulesets and enable immutable releases where available (SCM-02).

- ☐ Apply a seven-day minimum release age to dependency updates (DEP-02).
- ☐ Remove inactive members from organisations, registry scopes and marketplace namespaces, and revoke leavers' tokens from the inventory (ID-03, TPR-01).
- ☐ Subscribe to publish events for your own packages and to malicious-package feeds matched against your inventory (PUB-05, DET-02, DET-08).
- ☐ Adopt the supply chain playbook and run one 90-minute tabletop on the Trivy scenario (RSP-01).

14.3 Metrics

Metric	Target	Measured from
Publish paths using trusted publishing bound to a protected environment	100% by day 90	SEC-01 inventory, registry settings
Long-lived publish tokens remaining	0 by day 90	SEC-01 inventory
Static cloud keys in CI and on developer machines	0 by day 180	Secret scanning, cloud IAM reports
Third-party actions and CI images pinned by SHA or digest	100% by day 30	Workflow linter
Repositories with the workflow linter as a required check	100% by day 30	Source-host rulesets
Dependency installs through the internal proxy with release age enforced	≥ 95% by day 180	Proxy logs
Privileged accounts on phishing-resistant MFA	100% by day 14	Identity provider, source host, registries
Inactive members removed within 90 days of last activity	100% each quarter	Access review
Time to answer “are we affected?” for any package	< 15 minutes	Exercise and incident records
Time from detection to verified rotation of every reachable credential	< 24 hours	Incident records
Detection use cases 1–24 implemented and tested	9 by day 30, 24 by day 180	Detection engineering backlog
Tabletop exercises on real supply chain scenarios	2 per year	RSP-05 records

14.4 Common reasons programmes stall

The most common stall is treating trusted publishing as a registry setting rather than a pipeline redesign; it only works with the protected-environment and isolated-publish controls (SEC-04, CI-09). The second is making minimum release age optional, which lets urgent-looking updates bypass it; keep a documented security-fix path instead. The third is leaving developer endpoints out of scope because they belong to IT; the year's evidence makes them part of the build system.

CHAPTER 15

Master recommendation sheet

This sheet lists all 73 controls on one page set, for teams building a programme from scratch. Forty-four are priority P1, twenty-three are P2 and six are P3. Type is Preventive (P), Detective (D) or Responsive (R); Phase refers to Chapter 14; Owner follows Chapter 10.

15.1 If you can only do ten things

1. Replace every long-lived publish token with trusted publishing bound to a protected environment (SEC-02, SEC-04).
2. Stream source-control audit logs and alert on tag rewrites, new workflow files, visibility changes, runner registration and unsigned pushes followed by releases (DET-01).
3. Enforce a seven-day minimum release age on dependency updates (DEP-02).
4. Disable dependency install scripts by default (DEP-03).
5. Keep a complete credential inventory and rehearse atomic rotation from it (SEC-01, SEC-07).
6. Pin actions and images by SHA or digest and protect release tags (CI-05, SCM-02, ART-01).
7. Ban privileged workflows that run untrusted pull-request code, and lint every workflow (CI-01, CI-02).
8. Publish only from an isolated, approval-gated job that runs third-party tools nowhere near its secrets (CI-09, CI-10, PUB-02).
9. Put phishing-resistant MFA on every account that can push or publish (ID-01).
10. Monitor every package you publish for releases that did not come from your pipeline (PUB-05).

15.2 The full sheet

ID	Control	Type	Priority	Phase	Owner
ID-01	Phishing-resistant MFA on every privileged account	P	P1	0	IT and identity
ID-02	Separate identities for privileged publishing	P	P2	3	IT and identity
ID-03	Access reviews with automatic expiry and change alerts	P	P1	1	IT and identity
ID-04	Least-privilege, time-bound write access	P	P2	3	AppSec
ID-05	Governed service and bot identities	P	P1	0	Platform engineering
SCM-01	Rulesets on default and release branches, no silent bypass	P	P1	2	AppSec
SCM-02	Tag protection and immutable releases	P	P1	1	AppSec
SCM-03	CODEOWNERS on workflows, build scripts and agent configuration	P	P1	2	AppSec
SCM-04	Locked visibility, repository creation and release-pattern branches	P	P2	3	AppSec
SCM-05	Hidden-character detection	P	P2	3	AppSec
SCM-06	Token and OAuth-app policy for the source host	P	P1	2	AppSec
CI-01	No privileged workflows that run untrusted code	P	P1	1	Platform engineering
CI-02	No interpolation of untrusted context into scripts	P	P1	1	Platform engineering
CI-03	Least-privilege job tokens	P	P1	0	Platform engineering
CI-04	No secrets for forked contributions	P	P1	0	Platform engineering

ID	Control	Type	Priority	Phase	Owner
CI-05	SHA-pinned, allow-listed actions and CI images	P	P1	1	Platform engineering
CI-06	Cache isolation for release jobs	P	P1	2	Platform engineering
CI-07	Ephemeral runners and controlled runner registration	P	P1	2	Platform engineering
CI-08	Egress control on runners	P	P2	3	Platform engineering
CI-09	Isolated, approved publish step	P	P1	2	Platform engineering
CI-10	Third-party tools run without secrets	P	P1	2	Platform engineering
SEC-01	Complete credential inventory	P	P1	0	Security operations
SEC-02	Trusted publishing bound to ref and environment	P	P1	1-2	Platform engineering
SEC-03	Short-lived cloud credentials through OIDC	P	P1	2	Platform engineering
SEC-04	Secrets bound to protected contexts	P	P1	2	Platform engineering
SEC-05	Secret scanning with push protection	P	P2	0 / 4	AppSec
SEC-06	No long-lived tokens on developer machines	P	P1	1	IT and identity
SEC-07	Atomic rotation runbook	R	P1	2	Security operations
SEC-08	Signing keys in hardware or keyless signing	P	P2	4	Platform engineering
DEP-01	Lockfiles and frozen installs	P	P1	0	AppSec
DEP-02	Minimum release age	P	P1	1	AppSec
DEP-03	Install scripts off by default	P	P1	0	AppSec
DEP-04	Internal proxy registry with quarantine	P	P2	3	Platform engineering
DEP-05	New-dependency and maintainer-change review	P	P2	3	AppSec
DEP-06	Build-time SBOM and live inventory	D	P1	0	AppSec
DEP-07	Provenance verification with expected ref	P	P2	3	AppSec
DEP-08	Every execution point treated as code	P	P2	3	AppSec
PUB-01	Publish only from the release pipeline	P	P1	2	Platform engineering
PUB-02	Two-person release approval	P	P1	2	Platform engineering
PUB-03	Provenance, signed tags and immutable releases	P	P2	3	Platform engineering
PUB-04	Package-scoped publishing rights	P	P2	3	AppSec
PUB-05	Monitoring of your own releases	D	P1	1	Security operations
PUB-06	Same controls for stores and marketplaces	P	P1	2	Platform engineering
PUB-07	Reproducible builds and published-artefact verification	P	P3	4	Platform engineering
ART-01	Pull by digest and verify signatures	P	P1	2	Platform engineering
ART-02	Registry push only from the release pipeline	P	P1	2	Platform engineering
ART-03	Signed, verifiable release assets	P	P2	3	Platform engineering
END-01	Managed endpoints with developer-aware EDR	P	P1	2	IT and identity
END-02	Extension allow-list and staged updates	P	P1	2	IT and identity
END-03	AI-agent and workspace-trust policy	P	P1	2	IT and identity, AppSec
END-04	Sandboxed installs for untrusted code	P	P2	3	IT and identity
END-05	No vaults, wallets or browser passwords within reach	P	P2	3	IT and identity
END-06	Backups of developer home directories	R	P3	4	IT and identity
TPR-01	Token-driven offboarding	P	P1	1	IT and identity
TPR-02	OAuth and integration governance	P	P2	3	IT and identity
TPR-03	Contractual supply chain obligations	P	P3	4	Legal and procurement
TPR-04	Supplier and project health scoring	P	P3	4	AppSec
DET-01	Source-control audit-log alerting	D	P1	0-1	Security operations

ID	Control	Type	Priority	Phase	Owner
DET-02	Registry publish monitoring	D	P1	1	Security operations
DET-03	CI runtime telemetry	D	P2	3	Platform engineering, SecOps
DET-04	Developer-endpoint detections	D	P1	2	Security operations
DET-05	Honeytokens	D	P2	2	Security operations
DET-06	Cloud control-plane detections for build and developer identities	D	P2	3	Security operations
DET-07	Retired-credential watch for 90 days	D	P1	1	Security operations
DET-08	Intelligence matched to inventory	D	P1	1	Security operations
DET-09	Exfiltration-channel detection	D	P2	3	Security operations
DET-10	Brand and look-alike domain monitoring	D	P3	4	Security operations
RSP-01	Supply-chain-specific incident playbook	R	P1	0	CISO
RSP-02	Snapshot and isolate before revoking	R	P1	2	Security operations
RSP-03	Out-of-band coordination and private advisories	R	P2	3	CISO
RSP-04	"Did we ship it?" check	R	P1	2	CISO, Platform engineering
RSP-05	Exercises against real incidents	R	P2	3	CISO
RSP-06	Guardrails for AI-assisted analysis	R	P3	4	Security operations

15.3 Using the sheet

Copy the sheet into your tracker, add a status and a target date per row, and review it monthly. A control counts as done only when it is enforced by policy or configuration across every relevant system and verified by a test, such as a seeded `pull_request_target` workflow that the linter must reject or a canary token whose use must page someone.

Appendices

APPENDIX A

Incident timeline

The 38 events in this book, from 26 August 2025 to 23 September 2026, are plotted below by attack surface and campaign family. Numbers on the chart follow date order; milestones (diamonds) are the April trusted-publishing adoption, the July government advisory and the August arrests.

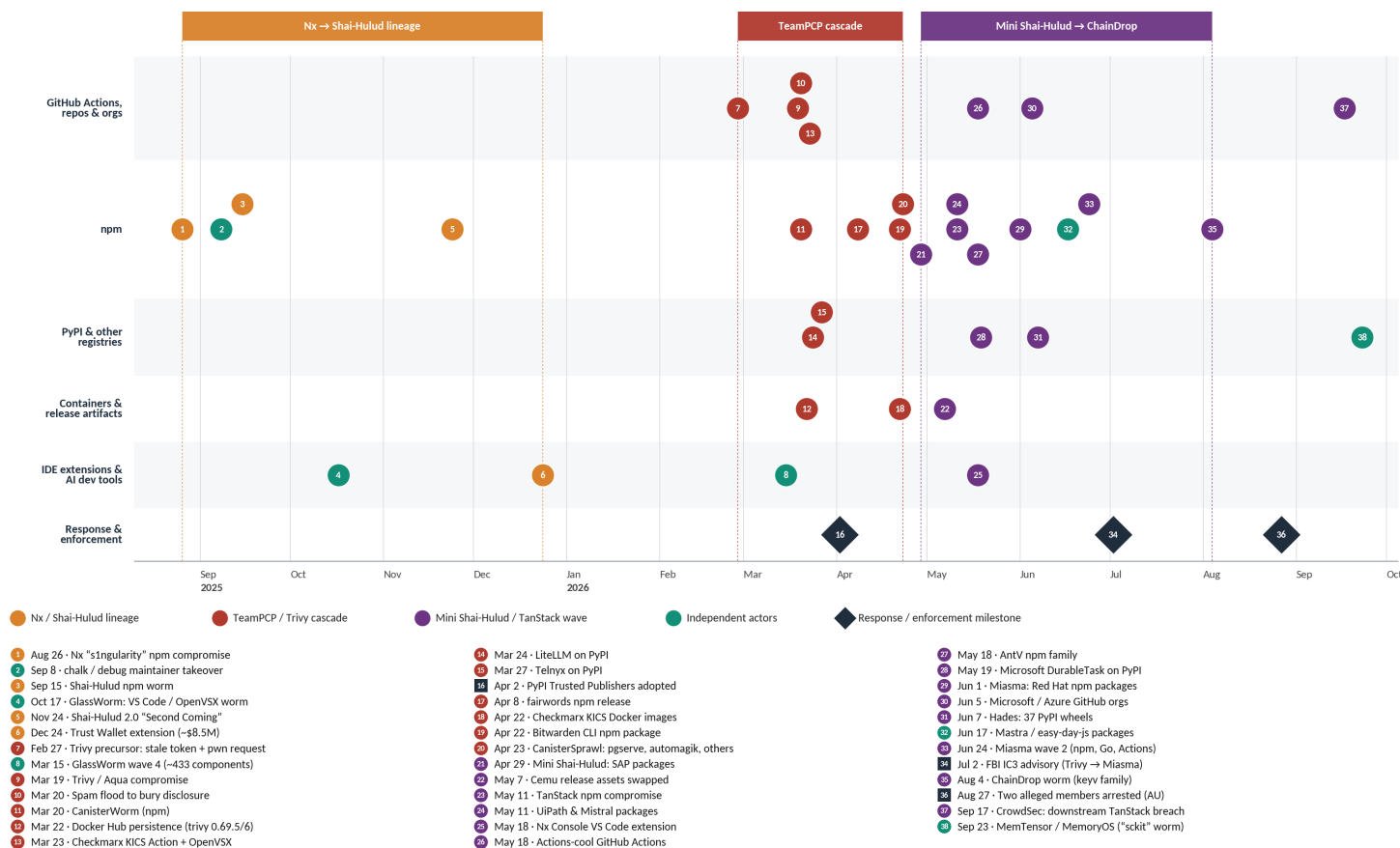


Figure A.1. 38 events (35 incidents and 3 response milestones) from August 2025 to September 2026, by attack surface (rows) and campaign family (colour), with a numbered key.

APPENDIX B

Glossary

Term	Meaning
Attestation	A signed statement about an artefact, such as how and where it was built
binding.gyp	Build configuration for native Node.js modules; can run commands during install
build.rs	A Rust build script that runs during cargo build
CODEOWNERS	A source-host file that requires review from named owners for matching paths
Credential relay	A campaign in which secrets stolen from one victim are used to compromise the next
Digest	A content hash that identifies an image or artefact immutably, unlike a tag
Ephemeral runner	A CI machine used for one job and then destroyed
EtherHiding	Storing command-and-control addresses in a blockchain smart contract so they cannot be taken down
Honeytoken	A fake credential planted to detect theft when it is used
Immutable release	A release whose tag and assets cannot be changed after publication
Install script	Code a package manager runs automatically during install, such as preinstall or postinstall
Lockfile	A file recording the exact versions and hashes of every dependency
Minimum release age	A rule that refuses dependency versions younger than a set period
OIDC (in CI)	A short-lived identity token issued to a CI job, exchanged for registry or cloud access
.pth file	A Python path file that can execute code every time the interpreter starts
Provenance	Metadata stating which source, workflow and builder produced an artefact
pull_request_target	A GitHub Actions trigger that runs in the base repository's privileged context for pull requests, including from forks
Pwn request	An attack that abuses a privileged pull-request workflow to run attacker code with repository secrets
Ruleset	A source-host policy that protects branches or tags, for example blocking force-pushes
SBOM	Software bill of materials: a list of the components in a build
SLSA	Supply-chain Levels for Software Artifacts, a framework for build integrity and provenance
Trusted publishing	Publishing to a registry with a short-lived OIDC exchange instead of a stored token
Workspace Trust	An editor feature that restricts what a newly opened folder can execute
Worm	Malware that spreads on its own, here by republishing packages with stolen tokens

APPENDIX C

Mapping to common frameworks

The control domains map to established frameworks at the practice-group level. Use the mapping to show auditors where this programme satisfies existing obligations; consult each framework for exact requirements.

Domain	NIST SSDF (SP 800-218)	SLSA v1	OpenSSF S2C2F	CIS Software Supply Chain Security Guide	ISO/IEC 27001:2022 Annex A
ID: Identity and access	PO.5, PS.1	Source integrity	n/a	Source code	5.15–5.18, 8.2, 8.5
SCM: Source control	PS.1, PS.3	Source integrity	n/a	Source code	8.4, 8.32
CI: CI/CD hardening	PO.3, PO.5	Build L2–L3 (hardened, isolated builds)	Rebuild	Build pipelines	8.25, 8.31
SEC: Secrets and credentials	PO.5, PS.1	Build L3 (secret isolation)	n/a	Build pipelines	5.17, 8.24
DEP: Dependency intake	PW.4, RV.1	Verifying dependencies' provenance	Ingest, Scan, Inventory, Update, Enforce	Dependencies	5.19–5.21, 8.8
PUB: Publishing	PS.2, PS.3	Build L2–L3 provenance	n/a	Artifacts	8.25, 8.32
ART: Artefacts	PS.2	Provenance verification	Enforce	Artifacts, Deployment	8.32
END: Endpoints and tools	PO.5	n/a	n/a	n/a	8.1, 8.7
TPR: Third parties	PO.1	n/a	Audit	Dependencies	5.19–5.23
DET: Detection	RV.1	n/a	Audit	All domains	8.15, 8.16
RSP: Response	RV.2, RV.3	n/a	Fix and upstream	n/a	5.24–5.28

APPENDIX D

Sources and further reading

The incident record combines the authors' own incident notes with public reporting. The pages below were identified during research for specific incidents; read them for indicators and full technical detail, and check for later revisions.

- Datadog Security Labs, [ChainDrop worm compromises hundreds of popular npm packages](https://securitylabs.datadoghq.com/articles/npm-worm-compromises-popular-npm-packages/) <https://securitylabs.datadoghq.com/articles/npm-worm-compromises-popular-npm-packages/>
- Zscaler ThreatLabz, [Tracking Shai-Hulud: inside the ChainDrop npm worm](https://www.zscaler.com/blogs/security-research/tracking-shai-hulud-inside-chaindrop-npm-worm) <https://www.zscaler.com/blogs/security-research/tracking-shai-hulud-inside-chaindrop-npm-worm>
- Aikido Security, [Keyv and friends compromised in npm supply chain attack](https://www.aikido.dev/blog/keyv-and-friends-compromised-in-npm-supply-chain-attack) <https://www.aikido.dev/blog/keyv-and-friends-compromised-in-npm-supply-chain-attack>
- Aikido Security, [MemTensor npm and PyPI hit by “supplychain.local” malware](https://www.aikido.dev/blog/supplychain-local-memtensor-npm-pypi) <https://www.aikido.dev/blog/supplychain-local-memtensor-npm-pypi>
- Semgrep, [Inside the MemTensor compromise](https://semgrep.dev/blog/2026/the-ai-ecosystem-has-worms-now-inside-the-memtensor-compromise/) <https://semgrep.dev/blog/2026/the-ai-ecosystem-has-worms-now-inside-the-memtensor-compromise/>
- SafeDep, [Threat intelligence campaigns](https://safedep.io/ti/campaigns/) <https://safedep.io/ti/campaigns/>
- StepSecurity, [The state of open source supply chain attacks](https://www.stepsecurity.io/blog/state-of-open-source-supply-chain-attacks) <https://www.stepsecurity.io/blog/state-of-open-source-supply-chain-attacks>
- StepSecurity, [Five supply chain attacks in 48 hours](https://www.stepsecurity.io/blog/5-supply-chain-attacks-in-48-hours-why-securing-one-layer-is-not-enough) <https://www.stepsecurity.io/blog/5-supply-chain-attacks-in-48-hours-why-securing-one-layer-is-not-enough>
- Cloud Security Alliance, [Single points of failure: a week of supply chain compromises](https://labs.cloudsecurityalliance.org/research/csa-research-note-supply-chain-single-points-of-failure-2026/) <https://labs.cloudsecurityalliance.org/research/csa-research-note-supply-chain-single-points-of-failure-2026/>
- Cloud Security Alliance, [GlassWorm returns: developer toolchain worm expands](https://labs.cloudsecurityalliance.org/research/csa-research-note-glassworm-supply-chain-developer-toolchain/) <https://labs.cloudsecurityalliance.org/research/csa-research-note-glassworm-supply-chain-developer-toolchain/>
- TuxCare, [The software supply chain broke in 2026](https://tuxcare.com/blog/supply-chain-broke-in-2026/) <https://tuxcare.com/blog/supply-chain-broke-in-2026/>
- Upwind, [npm malware built for CI/CD and cloud compromise](https://www.upwind.io/feed/large-scale-npm-pypi-supply-chain-campaign-suspected-across-multiple-popular-packages) <https://www.upwind.io/feed/large-scale-npm-pypi-supply-chain-campaign-suspected-across-multiple-popular-packages>
- Dark Reading, [Supply chain worms in 2026: what Shai-Hulud taught attackers](https://www.darkreading.com/cyberattacks-data-breaches/supply-chain-worms-in-2026-what-shai-hulud-taught-attackers-and-how-to-prepare) <https://www.darkreading.com/cyberattacks-data-breaches/supply-chain-worms-in-2026-what-shai-hulud-taught-attackers-and-how-to-prepare>
- The Hacker News, [Self-spreading GlassWorm infects VS Code extensions](https://thehackernews.com/2025/10/self-spreading-glassworm-infects-vs.html) <https://thehackernews.com/2025/10/self-spreading-glassworm-infects-vs.html>
- The Hacker News, [GlassWorm returns with 24 malicious extensions](https://thehackernews.com/2025/12/glassworm-returns-with-24-malicious.html) <https://thehackernews.com/2025/12/glassworm-returns-with-24-malicious.html>
- The Hacker News, [Trust Wallet Chrome extension hack](https://thehackernews.com/2025/12/trust-wallet-chrome-extension-hack.html) <https://thehackernews.com/2025/12/trust-wallet-chrome-extension-hack.html>
- Socket, [GlassWorm loader hits Open VSX via developer account compromise](https://socket.dev/blog/glassworm-loader-hits-open-vsx-via-suspected-developer-account-compromise) <https://socket.dev/blog/glassworm-loader-hits-open-vsx-via-suspected-developer-account-compromise>
- Truesec, [GlassWorm: self-propagating VS Code extension worm](https://www.truesec.com/hub/blog/glassworm-self-propagating-vscode-extension) <https://www.truesec.com/hub/blog/glassworm-self-propagating-vscode-extension>
- Public timeline repository, [TeamPCP supply chain attack](https://github.com/ugurates/teampcp-supply-chain-attack) <https://github.com/ugurates/teampcp-supply-chain-attack>

Frameworks referenced: NIST SP 800-218 (SSDF), SLSA v1, OpenSSF S2C2F, the CIS Software Supply Chain Security Guide and ISO/IEC 27001:2022.

**“A credential stolen today is used within eight days.
Some attackers wait five weeks.”**

Between August 2025 and September 2026, trusted software became the most efficient way to steal credentials at scale. A single injectable workflow seeded the first self-replicating npm worm. One stolen bot token turned a security scanner into a credential vending machine across GitHub Actions, Docker Hub, npm and PyPI. Short-lived OIDC tokens were lifted from runner memory, and malicious releases shipped with valid provenance.

Breaking the Chain reconstructs 35 of those incidents in a single, comparable format (what happened, the entry point, the payload, the defences it walked past and the controls that would have stopped it) and turns the record into a programme any organisation can build from scratch.

INSIDE

- Every incident from Nx “s1ngularity” and Shai-Hulud to TeamPCP, TanStack, Miasma, ChainDrop and MemTensor
- Eighteen prevention methods attackers bypassed, and why
- An incident-to-control matrix ranking the controls missed most often
- 73 preventive, detective and responsive controls with platform-level guidance
- 24 detection use cases, a response model and a phased 365-day roadmap

